Sovereign Yield Curve Construction & Modelling : la courbe des taux souveraine en Mauritanie (MRU)

Auteur : Kader, MSc Financial Markets, Trading & Risk Management (INSEEC)

Objectif : construire la courbe des taux de l'État mauritanien à partir des taux d'adjudication publiés par la Banque Centrale de Mauritanie (BCM), étudier comment elle bouge, s'en servir pour valoriser des titres publics et mesurer leur risque de taux, puis simuler l'évolution des taux avec un modèle de Vasicek.

Données : fichier de la Banque Centrale de Mauritanie `data/Marche_Monetaire.xlsx`, onglet « Marché monétaire » : 196 relevés hebdomadaires du 06/01/2022 au 12/02/2026, avec le taux directeur, les deux facilités (dépôt et prêt) et les taux de 11 maturités de titres du Trésor (bons à 4, 13, 26 et 52 semaines ; obligations à 2, 3, 4, 5, 8, 9 et 10 ans). Aucune autre donnée n'est utilisée.

Plan :

1. Préparation et contrôle des données
2. Construction de la courbe : interpolation linéaire, spline cubique, Nelson-Siegel ; comparaison ; taux zéro-coupon, facteurs d'actualisation, forwards
3. Dynamique de la courbe : analyse en composantes principales (ACP)
4. Valorisation et risque de taux : prix, duration, convexité, DV01, key rate durations, scénarios de choc
5. Modèle de Vasicek et Monte Carlo : calibration, simulation, validation, VaR
6. Limites : une courbe construite à partir des adjudications
7. Synthèse

In [1]:
```
# Bibliothèques utilisées
import numpy as np                              # calculs numériques
import pandas as pd                             # tableaux de données
import matplotlib.pyplot as plt                 # graphiques
import statsmodels.api as sm                    # régression (calibration de
Vasicek)
from scipy.interpolate import CubicSpline       # spline cubique
from scipy.optimize import brentq               # résolution d'équation
(taux actuariel d'un titre)

# Réglages d'affichage
plt.rcParams["figure.figsize"] = (11, 4)
plt.rcParams["axes.grid"] = True
plt.rcParams["grid.color"] = "#e5e5e5"
plt.rcParams["axes.spines.top"] = False
plt.rcParams["axes.spines.right"] = False
pd.options.display.float_format = "{:.3f}".format

# Couleurs fixes pour tout le notebook
```

```
BLEU = "#2a78d6"      # interpolation linéaire / courbe de référence
ORANGE = "#eb6834"    # spline cubique
VERT = "#1f9e74"      # Nelson-Siegel
VIOLET = "#7b5cc4"
GRIS = "#8a8a8a"

import warnings
warnings.filterwarnings("ignore")   # masque des avertissements
techniques sans importance
```

1. Préparation et contrôle des données

1.1 Chargement

Le tableau commence à la ligne 4 de l'onglet (ligne des titres) et occupe les colonnes B à P. La colonne Q (moyenne pondérée) et les tableaux de synthèse sous la ligne 200 ne sont pas des données brutes : on ne les lit pas.

In [2]:
```
brut = pd.read_excel("data/Marche_Monetaire.xlsx", sheet_name="Marché
monétaire",
                     header=3, usecols="B:P", nrows=196)
brut = brut.set_index("Date")

# Noms de colonnes courts
brut.columns = ["directeur", "depot", "pret",
                "4S", "13S", "26S", "52S", "2A", "3A", "4A", "5A", "8A",
"9A", "10A"]

print(len(brut), "semaines, du", brut.index.min().date(), "au",
brut.index.max().date())
brut.tail()
```

```
196 semaines, du 2022-01-06 au 2026-02-12
```

Out[2]:

Date	directeur	depot	pret	4S	13S	26S	52S	2A	3A	4A	5A	8A	9A	10A
2026-01-15	6.000	2	6.500	2.040	2.730	4.420	3.070	3.930	5.170	4.920	4.670	3.710	3.840	4.150
2026-01-22	6.000	2	6.500	2.040	3.600	4.260	4.370	3.930	5.170	4.920	4.670	3.710	3.840	4.150
2026-01-29	6.000	2	6.500	2.040	3.600	4.260	4.370	3.930	5.490	4.920	4.670	3.710	3.840	4.150
2026-02-05	6.000	2	6.500	2.040	4.120	4.340	3.560	3.930	5.490	4.920	4.670	3.710	3.840	4.150
2026-02-12	6.000	2	6.500	2.040	4.120	4.340	3.560	3.930	5.490	4.920	4.670	3.710	3.840	4.150

Les taux sont en %. Les trois premières colonnes sont les taux de la politique monétaire (taux directeur, facilité de dépôt, facilité de prêt). Les 11 suivantes sont les taux des titres du Trésor, du plus court (bon à 4 semaines) au plus long (obligation à 10 ans).

On garde aussi la maturité de chaque titre **en années**, ce qui sert dans tous les calculs.

In [3]:
```
NOMS = ["4S", "13S", "26S", "52S", "2A", "3A", "4A", "5A", "8A", "9A",
"10A"]
MATURITES = pd.Series([4/52, 13/52, 26/52, 1, 2, 3, 4, 5, 8, 9, 10],
index=NOMS)   # en années
```

```
politique = brut[["directeur", "depot", "pret"]]   # taux de la BCM, en
%
taux_brut = brut[NOMS]                             # taux des titres, en
%
MATURITES.round(3)
```

Out[3]:
```
4S     0.077
13S    0.250
26S    0.500
52S    1.000
2A     2.000
3A     3.000
4A     4.000
5A     5.000
8A     8.000
9A     9.000
10A   10.000
dtype: float64
```

1.2 Contrôle qualité

On vérifie : les dates (ordre, doublons, écart entre deux relevés), les valeurs manquantes par maturité, et les taux répétés d'une semaine à l'autre.

In [4]:
```
ecart_jours = brut.index.to_series().diff().dt.days
print("Dates triées :", brut.index.is_monotonic_increasing, "| doublons
:", brut.index.duplicated().sum())
print("Jour de la semaine :",
brut.index.day_name().value_counts().to_dict())
print("Écart entre deux relevés (jours) :",
ecart_jours.value_counts().sort_index().to_dict())
```

```
Dates triées : True | doublons : 0
Jour de la semaine : {'Thursday': 194, 'Friday': 1, 'Wednesday': 1}
Écart entre deux relevés (jours) : {6.0: 2, 7.0: 182, 8.0: 2, 14.0: 5, 21.
0: 2, 42.0: 2}
```

Interprétation : les relevés sont faits le jeudi (194 sur 196, un mercredi et un vendredi sont sans doute des jours fériés). L'écart normal est de 7 jours, mais il y a **9 trous de 2 à 6 semaines**, surtout en 2022-2023. On ne les comble pas : ce serait inventer des taux. On en tient compte quand on travaille sur des variations hebdomadaires (section 5).

In [5]:
```
# Pour chaque maturité : première date disponible, nombre de valeurs,
nombre de taux différents
controle = pd.DataFrame({
    "maturité (ans)": MATURITES,
    "première date": [taux_brut[c].first_valid_index().date() for c in
NOMS],
    "nb de valeurs": taux_brut.notna().sum(),
    "nb de taux différents": taux_brut.nunique(),
    "% semaines = semaine d'avant":
[round((taux_brut[c].dropna().diff().dropna() == 0).mean() * 100) for c
in NOMS],
    "min (%)": taux_brut.min(),
    "max (%)": taux_brut.max(),
```

```
})
controle
```

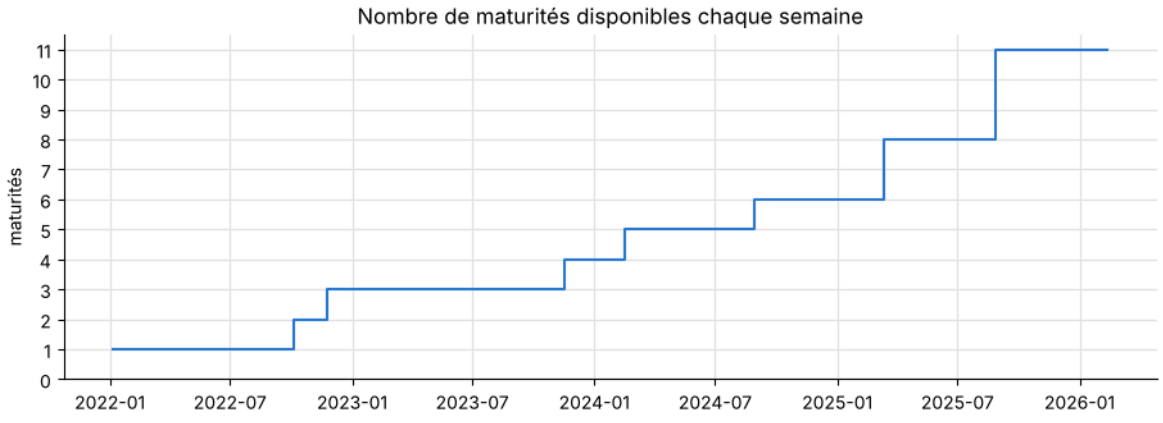
Out[5]:

	maturité (ans)	première date	nb de valeurs	nb de taux différents	% semaines = semaine d'avant	min (%)	max (%)
4S	0.077	2022-10-06	172	141	15	1.960	8.070
13S	0.250	2023-11-16	118	63	42	2.120	8.190
26S	0.500	2022-01-06	196	104	42	1.463	8.130
52S	1.000	2022-11-24	167	14	92	3.070	7.840
2A	2.000	2024-02-15	105	4	97	3.930	8.150
3A	3.000	2024-08-29	77	4	96	4.430	7.010
4A	4.000	2025-03-13	49	1	100	4.920	4.920
5A	5.000	2025-03-13	49	1	100	4.670	4.670
8A	8.000	2025-08-28	25	1	100	3.710	3.710
9A	9.000	2025-08-28	25	1	100	3.840	3.840
10A	10.000	2025-08-28	25	1	100	4.150	4.150

In [6]:
```
# Combien de maturités sont disponibles chaque semaine ?
nb_maturites = taux_brut.notna().sum(axis=1)

fig, ax = plt.subplots(figsize=(11, 3.5))
ax.step(nb_maturites.index, nb_maturites, color=BLEU, where="post")
ax.set_title("Nombre de maturités disponibles chaque semaine")
ax.set_ylabel("maturités")
ax.set_yticks(range(0, 12))
plt.show()

for k in [4, 5, 11]:
    print(f"Semaines avec au moins {k} maturités :", (nb_maturites >=
k).sum())
```




```
Semaines avec au moins 4 maturités : 118
Semaines avec au moins 5 maturités : 105
Semaines avec au moins 11 maturités : 25
```

Ce que montre le contrôle :

1. **Les maturités arrivent progressivement.** En 2022, seul le bon à 26 semaines est disponible en continu. Le marché s'allonge ensuite : 4S fin 2022, 13S fin 2023, 2 ans début 2024, 3 ans mi-2024, 4 et 5 ans en mars 2025, 8, 9 et 10 ans fin août 2025. La courbe complète (11 maturités) n'existe que sur **25 semaines**.
2. **Les taux sont reportés.** Le fichier répète le dernier taux d'adjudication tant qu'il n'y a pas de nouvelle émission. Les obligations à 4, 5, 8, 9 et 10 ans n'ont

qu'une seule valeur sur toute la période (une seule émission) ; le 2 et le 3 ans en ont 4 ; le 52S est identique à la semaine précédente dans 92 % des cas.

3. **Valeurs manquantes** : avant l'arrivée de chaque maturité, la case est vide. On ne les remplit pas.

Décisions :

- la courbe est construite chaque semaine qui a **au moins 5 maturités** (105 semaines), car Nelson-Siegel a 4 paramètres : il faut au moins 5 points pour qu'il reste une erreur à mesurer ;
- la valorisation et les chocs utilisent la **dernière courbe complète** (12/02/2026) ;
- on garde tous les taux observés, sans en supprimer ni en compléter.

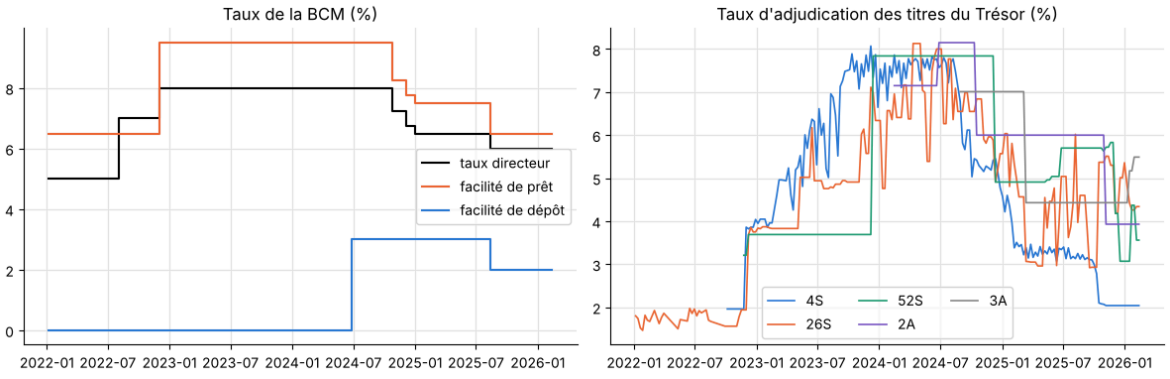
1.3 Visualisation des taux

On affiche les taux de la politique monétaire (le « corridor ») et quelques maturités représentatives.

In [7]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].step(politique.index, politique["directeur"], color="black",
where="post", label="taux directeur")
axes[0].step(politique.index, politique["pret"], color=ORANGE,
where="post", label="facilité de prêt")
axes[0].step(politique.index, politique["depot"], color=BLEU,
where="post", label="facilité de dépôt")
axes[0].set_title("Taux de la BCM (%)")
axes[0].legend()

for nom, couleur in [("4S", BLEU), ("26S", ORANGE), ("52S", VERT),
("2A", VIOLET), ("3A", GRIS)]:
    axes[1].plot(taux_brut.index, taux_brut[nom], color=couleur,
linewidth=1.2, label=nom)
axes[1].set_title("Taux d'adjudication des titres du Trésor (%)")
axes[1].legend(ncol=3)
plt.tight_layout()
plt.show()
```



Interprétation :

- La BCM a **resserré** sa politique en 2022 (taux directeur de 5 % à 8 %), puis l'a **assouplie** à partir de fin 2024 (de 8 % à 6 % en août 2025). Le taux de dépôt est à 0 jusqu'en juin 2024.
- **Anomalie** : d'août à novembre 2022, la facilité de prêt (6,5 %) est sous le taux directeur (7 puis 8 %). Normalement, le prêt est au-dessus du directeur ; il s'agit soit d'un décalage de mise à jour dans le fichier, soit d'une période de transition. Cela ne touche pas la courbe des titres.
- Les taux des titres suivent le cycle de la BCM (hausse jusqu'à 8 % en 2024, baisse vers 2-4 % en 2025-2026), mais avec **des sauts de 1,5 à 2 points d'une adjudication à l'autre** (par exemple le 26S : 6,02 % puis 3,97 % en août 2025 ; le 13S tombe à 4,86 % le 15/02/2024 entre 6,84 et 7,91). Ce bruit est typique d'un marché d'adjudication peu profond : le taux dépend de la demande des banques le jour de l'émission. On garde ces valeurs, ce sont des résultats réels.
- Depuis la baisse des taux, beaucoup de titres s'échangent **sous le taux directeur** : la liquidité bancaire est abondante et les banques placent leurs excédents en titres publics.

1.4 Une seule convention de taux

Les bons (≤ 1 an) et les obligations (≥ 2 ans) ne sont pas cotés de la même façon :

- un **bon** est un titre sans coupon, coté en **taux simple** : on paie P et on reçoit 100 à l'échéance, avec $P = 100/(1 + r \times T)$;
- une **obligation** paie un coupon annuel ; le taux d'adjudication est un **taux actuariel** (taux composé annuellement).

Pour comparer des choses comparables, on convertit les bons en **taux actuariel** : $y = (1 + r \times T)^{1/T} - 1$. La différence est petite (quelques points de base) mais elle rend la courbe cohérente.

Hypothèse : chaque obligation est émise au pair, donc son taux d'adjudication est aussi son coupon et c'est un **taux de rendement au pair**. Pour un titre de 1 an ou moins, il n'y a pas de coupon intermédiaire, donc taux au pair = taux zéro-coupon.

In [8]:
```
def taux_actuariel_bon(taux_simple, T):
    # Convertit le taux simple d'un bon (sans coupon) en taux actuariel
annuel
    return (1 + taux_simple * T) ** (1 / T) - 1

taux = taux_brut / 100                    # en décimal (0,05 = 5 %)
for nom in ["4S", "13S", "26S", "52S"]:
    taux[nom] = taux_actuariel_bon(taux[nom], MATURITES[nom])

print("Exemple 12/02/2026, 13 semaines : taux simple",
taux_brut.loc["2026-02-12", "13S"],
      "% -> taux actuariel", round(taux.loc["2026-02-12", "13S"] * 100,
3), "%")
```

```
Exemple 12/02/2026, 13 semaines : taux simple 4.12 % -> taux actuariel 4.1
84 %
```

2. Construction de la courbe

On dispose chaque semaine de quelques points (maturité, taux). Construire la courbe, c'est **relier ces points** pour obtenir un taux pour n'importe quelle maturité. On compare trois méthodes :

Méthode	Idée	Passe par tous les points ?
Interpolation linéaire	des segments de droite entre deux points voisins (méthode de la BCM)	oui
Spline cubique	des morceaux de polynômes de degré 3, raccordés de façon lisse	oui
Nelson-Siegel	une formule à 4 paramètres ajustée au mieux sur les points	non (lissage)

2.1 Interpolation linéaire et spline cubique

In [9]:
```python
def courbe_lineaire(T_obs, y_obs, T):
    # Segments de droite entre les points ; au-delà des points extrêmes,
le taux reste plat
    return np.interp(T, T_obs, y_obs)

def courbe_spline(T_obs, y_obs, T):
    # Spline cubique « naturelle » (courbure nulle aux deux extrémités)
    spline = CubicSpline(T_obs, y_obs, bc_type="natural")
    T = np.clip(T, T_obs[0], T_obs[-1])      # pas d'extrapolation :
plat au-delà des points
    return spline(T)
```

2.2 Nelson-Siegel

Le modèle de Nelson-Siegel décrit toute la courbe avec 4 paramètres :

$$y(T) = \beta_0 + \beta_1 \frac{1 - e^{-\lambda T}}{\lambda T} + \beta_2 \left(\frac{1 - e^{-\lambda T}}{\lambda T} - e^{-\lambda T} \right)$$

- β_0 = **niveau** : le taux très long terme (quand T devient grand, $y \to \beta_0$) ;
- β_1 = **pente** : $\beta_0 + \beta_1$ est le taux très court, donc $\beta_1 < 0$ signifie une courbe croissante ;
- β_2 = **courbure** : une bosse ($\beta_2 > 0$) ou un creux ($\beta_2 < 0$) au milieu de la courbe ;
- λ = **vitesse de décroissance** : elle fixe la maturité où la bosse est la plus marquée (environ $1{,}8/\lambda$ ans).

Méthode de calibration (simple et robuste) : pour un λ donné, la formule est **linéaire** en $\beta_0, \beta_1, \beta_2$: une simple régression par moindres carrés donne les trois β. On essaie donc une grille de valeurs de λ (de 0,3 à 3) et on garde celle qui donne la plus petite erreur.

Garde-fou : on n'accepte que des paramètres économiquement plausibles, avec un taux long (β_0) et un taux court ($\beta_0 + \beta_1$) entre 0 et 15 %. Sans ce garde-fou, avec seulement 5 ou 6 points, la régression peut donner des β absurdes (par

exemple $\beta_0 = -480$ % et $\beta_1 = +490$ %) qui se compensent : la courbe est correcte sur les points mais les paramètres n'ont plus de sens.

In [10]:
```
def facteurs_ns(T, lam):
    # Les trois « formes » de Nelson-Siegel pour des maturités T
(niveau, pente, courbure)
    T = np.atleast_1d(T)
    x = lam * T
    pente = (1 - np.exp(-x)) / x
    courbure = pente - np.exp(-x)
    return np.column_stack([np.ones_like(T), pente, courbure])

def courbe_ns(params, T):
    # Taux Nelson-Siegel pour des maturités T ; params = (beta0, beta1,
beta2, lambda)
    b0, b1, b2, lam = params
    return facteurs_ns(T, lam) @ np.array([b0, b1, b2])

GRILLE_LAMBDA = np.arange(0.3, 3.001, 0.05)

def calibrer_ns(T_obs, y_obs, grille=GRILLE_LAMBDA):
    # Pour chaque lambda : régression des taux sur les 3 facteurs ; on
garde la plus petite erreur
    meilleur = None
    for lam in grille:
        X = facteurs_ns(T_obs, lam)
        betas = np.linalg.lstsq(X, y_obs, rcond=None)[0]
        b0, b1, b2 = betas
        if not (0 < b0 < 0.15 and 0 < b0 + b1 < 0.15):   # garde-fou
            continue
        erreur = np.sum((X @ betas - y_obs) ** 2)
        if meilleur is None or erreur < meilleur[0]:
            meilleur = (erreur, [b0, b1, b2, lam])
    return meilleur[1]
```

2.3 Calibration semaine par semaine

On garde les 105 semaines avec au moins 5 maturités. Pour chacune, on calibre Nelson-Siegel et on note l'erreur d'ajustement (RMSE = racine de l'erreur quadratique moyenne, en points de base ; 1 pb = 0,01 %).

In [11]:
```
semaines = taux[taux.notna().sum(axis=1) >= 5]

def points_semaine(date):
    # Maturités et taux observés une semaine donnée (sans les cases
vides)
    ligne = semaines.loc[date].dropna()
    return MATURITES[ligne.index].values, ligne.values

lignes = []
for date in semaines.index:
    T_obs, y_obs = points_semaine(date)
    params = calibrer_ns(T_obs, y_obs)
    erreurs = courbe_ns(params, T_obs) - y_obs
    rmse_pb = np.sqrt(np.mean(erreurs ** 2)) * 10000
    lignes.append(params + [rmse_pb, len(T_obs)])
```

```
params_ns = pd.DataFrame(lignes, index=semaines.index,
                         columns=["beta0", "beta1", "beta2", "lambda",
"RMSE (pb)", "nb points"])
print(len(params_ns), "semaines calibrées, du",
params_ns.index.min().date(), "au", params_ns.index.max().date())
params_ns.tail()
```

```
105 semaines calibrées, du 2024-02-15 au 2026-02-12
```

Out[11]:

Date	beta0	beta1	beta2	lambda	RMSE (pb)	nb points
2026-01-15	0.018	0.006	0.090	0.450	53.706	11
2026-01-22	0.041	-0.027	0.052	3.000	42.757	11
2026-01-29	0.033	-0.009	0.069	0.950	48.419	11
2026-02-05	0.022	0.007	0.077	0.500	59.749	11
2026-02-12	0.022	0.007	0.077	0.500	59.749	11

2.4 Forme des courbes : dernière semaine (12/02/2026, 11 maturités)

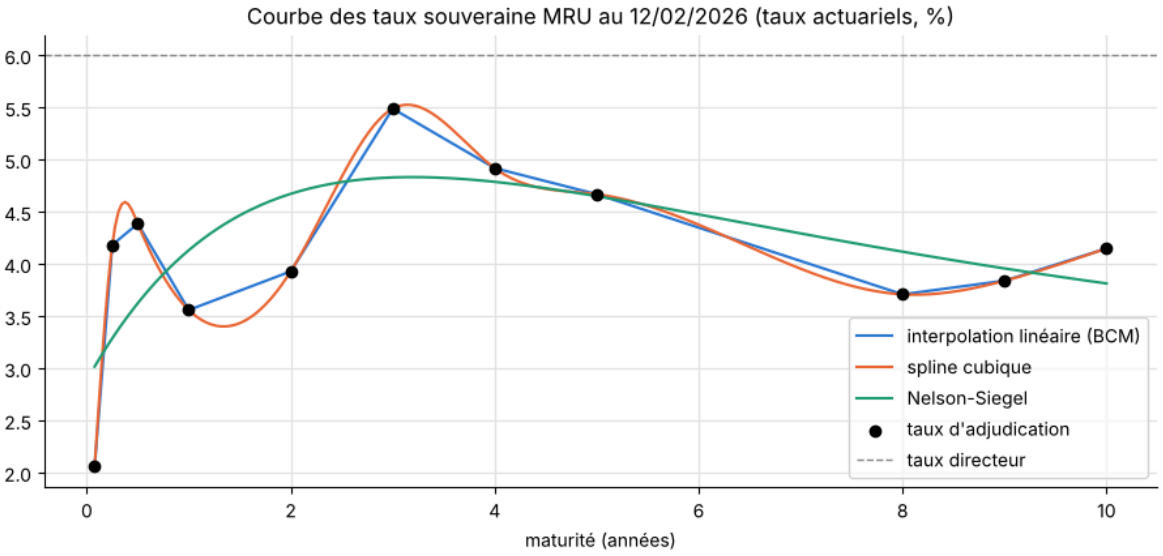
In [12]:

```
DATE_REF = pd.Timestamp("2026-02-12")
T_ref, y_ref = points_semaine(DATE_REF)
params_ref = params_ns.loc[DATE_REF, ["beta0", "beta1", "beta2",
"lambda"]].values

T_fin = np.linspace(4/52, 10, 300)     # maturités fines pour tracer les
courbes

fig, ax = plt.subplots(figsize=(11, 4.5))
ax.plot(T_fin, courbe_lineaire(T_ref, y_ref, T_fin) * 100, color=BLEU,
label="interpolation linéaire (BCM)")
ax.plot(T_fin, courbe_spline(T_ref, y_ref, T_fin) * 100, color=ORANGE,
label="spline cubique")
ax.plot(T_fin, courbe_ns(params_ref, T_fin) * 100, color=VERT,
label="Nelson-Siegel")
ax.scatter(T_ref, y_ref * 100, color="black", zorder=3, label="taux
d'adjudication")
ax.axhline(politique.loc[DATE_REF, "directeur"], color=GRIS,
linestyle="--", linewidth=1, label="taux directeur")
ax.set_title("Courbe des taux souveraine MRU au 12/02/2026 (taux
actuariels, %)")
ax.set_xlabel("maturité (années)")
ax.legend()
plt.show()

print("Paramètres Nelson-Siegel :", dict(zip(["beta0", "beta1", "beta2",
"lambda"], np.round(params_ref, 4))))
print("RMSE Nelson-Siegel :", round(params_ns.loc[DATE_REF, "RMSE
(pb)"], 1), "pb")
```




```
Paramètres Nelson-Siegel : {'beta0': np.float64(0.0219), 'beta1': np.float
64(0.0069), 'beta2': np.float64(0.0773), 'lambda': np.float64(0.5)}
RMSE Nelson-Siegel : 59.7 pb
```

Interprétation :

- La courbe observée est **très irrégulière** : 2 % à 4 semaines, environ 4,2-4,4 % de 3 mois à 6 mois, un creux à 1 an (3,6 %), un pic à 3 ans (5,5 %) puis une **baisse vers 3,7-4,2 % sur 8-10 ans**. Tous les titres sont sous le taux directeur (6 %).
- Cette forme ne vient pas d'anticipations de marché bien définies : les taux longs (8-10 ans) datent d'**une seule adjudication** en août 2025, le 3 ans d'une adjudication de janvier 2026. Chaque point a sa propre histoire.
- **Interpolation linéaire** : elle passe par tous les points et reste entre deux points voisins. C'est simple et transparent, d'où son usage à la BCM, mais la courbe a des « angles » (forwards en escalier, voir 2.7).
- **Spline cubique** : elle passe aussi par tous les points mais, pour rester lisse, elle **dépasse** entre les points : pic à environ 4,6 % vers 5 mois, creux à 3,4 % vers 16 mois, petit dépassement autour de 3 ans. Ces bosses ne correspondent à aucun taux observé.
- **Nelson-Siegel** : il ne passe pas par les points mais donne une forme simple (niveau, pente, une bosse). Il ne peut pas reproduire à la fois le creux à 1 an et le pic à 3 ans : l'erreur moyenne est d'environ 60 pb.

2.5 Comparaison des erreurs d'ajustement

Les deux interpolations passent par tous les points : leur erreur **sur les points utilisés** est nulle par construction, ce qui ne dit rien de leur qualité. Pour comparer honnêtement les trois méthodes, on fait une **validation croisée « leave-one-out »** :

1. on retire un point intérieur de la courbe (pas le plus court ni le plus long) ;
2. on construit la courbe avec les autres points ;
3. on regarde l'erreur entre le taux prédit et le vrai taux retiré.

On répète pour chaque point intérieur et chaque semaine. C'est une mesure de la capacité de la méthode à **deviner un taux qu'elle ne voit pas**, ce qui est exactement ce qu'on lui demande entre deux maturités cotées.

In [13]:
```
erreurs_loo = {"linéaire": [], "spline": [], "Nelson-Siegel": []}

for date in semaines.index:
    T_obs, y_obs = points_semaine(date)
    for i in range(1, len(T_obs) - 1):                    # points
intérieurs seulement
        T_sans, y_sans = np.delete(T_obs, i), np.delete(y_obs, i)
        vrai = y_obs[i]
        erreurs_loo["linéaire"].append(courbe_lineaire(T_sans, y_sans,
T_obs[i]) - vrai)
        erreurs_loo["spline"].append(courbe_spline(T_sans, y_sans,
T_obs[i]) - vrai)
        erreurs_loo["Nelson-
Siegel"].append(courbe_ns(calibrer_ns(T_sans, y_sans), T_obs[i])[0] -
vrai)

comparaison = pd.DataFrame({
    methode: {"nb de tests": len(e),
              "RMSE (pb)": np.sqrt(np.mean(np.square(e))) * 10000,
              "erreur absolue moyenne (pb)": np.mean(np.abs(e)) * 10000,
              "erreur max (pb)": np.max(np.abs(e)) * 10000}
    for methode, e in erreurs_loo.items()
}).T
comparaison["RMSE dans l'échantillon (pb)"] = [0, 0, params_ns["RMSE
(pb)"].mean()]
comparaison
```

Out[13]:

	nb de tests	RMSE (pb)	erreur absolue moyenne (pb)	erreur max (pb)	RMSE dans l'échantillon (pb)
linéaire	565.000	81.566	63.774	276.467	0.000
spline	565.000	121.147	87.665	573.170	0.000
Nelson-Siegel	565.000	89.448	71.039	280.127	43.333

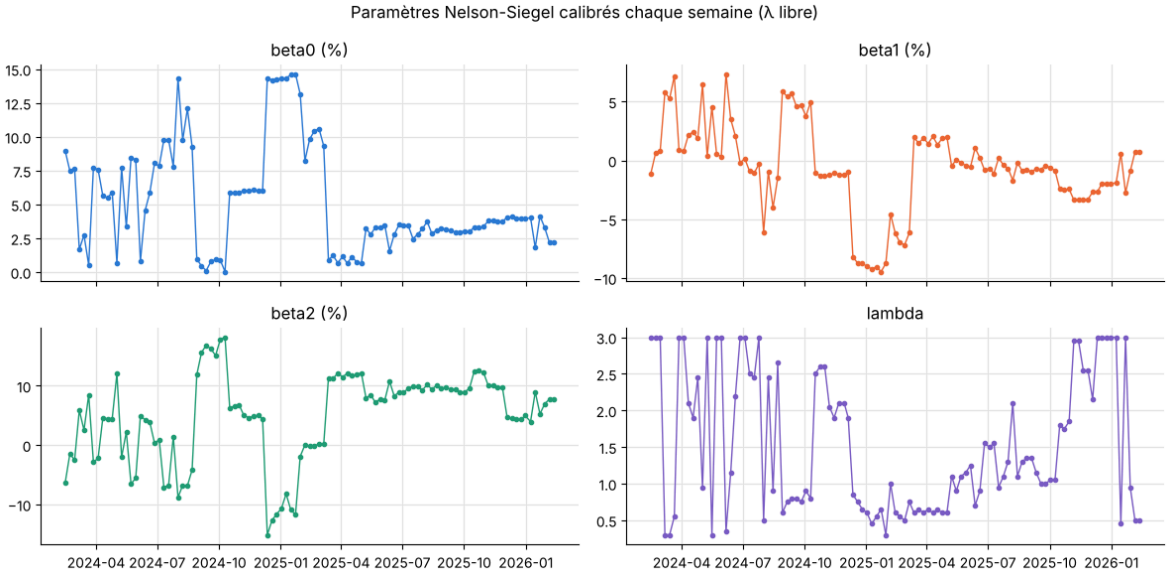
Interprétation :

- **L'interpolation linéaire prédit le mieux** les taux retirés (environ 82 pb d'erreur moyenne quadratique), devant Nelson-Siegel (environ 89 pb) et la spline cubique (environ 121 pb). Dans l'échantillon, Nelson-Siegel a une erreur moyenne de 43 pb (les interpolations 0 par construction).
- La **spline** est la pire : sur une courbe irrégulière avec de grands trous entre maturités, elle oscille et amplifie les erreurs.
- Les erreurs restent **grandes pour toutes les méthodes** (près de 1 %). Ce n'est pas un problème de méthode : les taux voisins ne sont pas cohérents entre eux, car ils viennent d'adjudications faites à des dates différentes. Aucune méthode ne peut deviner un taux qui n'a pas de lien avec ses voisins.
- **Conclusion pratique :** pour valoriser des titres, on garde l'**interpolation linéaire** comme courbe de référence (elle respecte les prix d'adjudication et c'est la méthode de la BCM). Nelson-Siegel sert à **résumer** la courbe en 3 facteurs et à étudier sa dynamique.

2.6 Stabilité des paramètres de Nelson-Siegel dans le temps

In [14]:
```
fig, axes = plt.subplots(2, 2, figsize=(12, 6), sharex=True)
for ax, nom, couleur in zip(axes.flat, ["beta0", "beta1", "beta2",
"lambda"], [BLEU, ORANGE, VERT, VIOLET]):
    serie = params_ns[nom] * (100 if nom != "lambda" else 1)
    ax.plot(serie.index, serie, color=couleur, marker=".", linewidth=1)
    ax.set_title(nom + (" (%)" if nom != "lambda" else ""))
plt.suptitle("Paramètres Nelson-Siegel calibrés chaque semaine (λ
libre)")
plt.tight_layout()
plt.show()

print("Part des semaines où lambda est collé à une borne de la grille
:",
      round(params_ns["lambda"].isin([GRILLE_LAMBDA.min(),
GRILLE_LAMBDA.max()]).mean() * 100), "%")
```




```
Part des semaines où lambda est collé à une borne de la grille : 20 %
```

Les paramètres sautent beaucoup d'une semaine à l'autre. Une solution classique (Diebold et Li, 2006) consiste à **fixer λ** une fois pour toutes et à ne recalibrer que les trois β chaque semaine. On prend pour λ la **médiane** des λ calibrés. On compare ensuite la stabilité : écart-type des variations hebdomadaires de chaque paramètre.

In [15]:
```
LAMBDA_FIXE = round(params_ns["lambda"].median(), 2)

lignes = []
for date in semaines.index:
    T_obs, y_obs = points_semaine(date)
    betas = np.linalg.lstsq(facteurs_ns(T_obs, LAMBDA_FIXE), y_obs,
rcond=None)[0]
    erreurs = facteurs_ns(T_obs, LAMBDA_FIXE) @ betas - y_obs
    lignes.append(list(betas) + [np.sqrt(np.mean(erreurs ** 2)) *
10000])
params_fixe = pd.DataFrame(lignes, index=semaines.index, columns=
["beta0", "beta1", "beta2", "RMSE (pb)"])
```

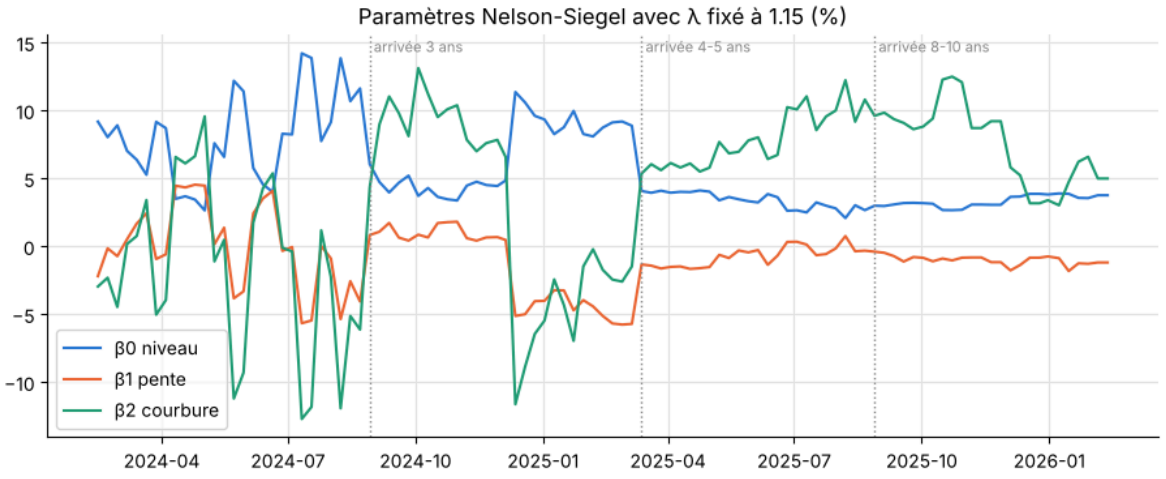
```
stabilite = pd.DataFrame({
    "λ libre": (params_ns[["beta0", "beta1", "beta2"]].diff().std() *
10000).tolist() + [params_ns["RMSE (pb)"].mean()],
    f"λ fixé à {LAMBDA_FIXE}": (params_fixe[["beta0", "beta1",
"beta2"]].diff().std() * 10000).tolist() + [params_fixe["RMSE
(pb)"].mean()],
}, index=["écart-type variation β0 (pb)", "écart-type variation β1
(pb)", "écart-type variation β2 (pb)", "RMSE moyen (pb)"])
stabilite
```

Out[15]:

	λ libre	λ fixé à 1.15
écart-type variation β0 (pb)	257.877	193.422
écart-type variation β1 (pb)	240.721	179.694
écart-type variation β2 (pb)	463.665	422.403
RMSE moyen (pb)	43.333	46.288

In [16]:

```
fig, ax = plt.subplots(figsize=(11, 4))
ax.plot(params_fixe.index, params_fixe["beta0"] * 100, color=BLEU,
label="β0 niveau")
ax.plot(params_fixe.index, params_fixe["beta1"] * 100, color=ORANGE,
label="β1 pente")
ax.plot(params_fixe.index, params_fixe["beta2"] * 100, color=VERT,
label="β2 courbure")
for date, nom in [("2024-08-29", "3 ans"), ("2025-03-13", "4-5 ans"),
("2025-08-28", "8-10 ans")]:
    ax.axvline(pd.Timestamp(date), color=GRIS, linestyle=":",
linewidth=1)
    ax.text(pd.Timestamp(date), ax.get_ylim()[1] * 0.92, " arrivée " +
nom, color=GRIS, fontsize=8)
ax.set_title(f"Paramètres Nelson-Siegel avec λ fixé à {LAMBDA_FIXE}
(%)")
ax.legend(loc="lower left")
plt.show()
```



Interprétation :

- Avec **λ libre**, λ se colle souvent aux bornes de la grille et les β font des sauts de plusieurs points de pourcentage : le modèle compense un λ différent par des β différents. Les paramètres ne sont **pas interprétables** d'une semaine à l'autre.
- Avec **λ fixé**, les variations hebdomadaires des β baissent d'environ 25 % pour β_0 et β_1 (et 10 % pour β_2), pour une erreur d'ajustement à peine plus élevée

(46 pb contre 43 pb). C'est le compromis classique : un peu moins de précision, des paramètres plus stables et comparables dans le temps.

- Le graphique montre surtout que **la stabilité dépend des maturités disponibles**. Jusqu'en mars 2025, la courbe s'arrête à 2 ou 3 ans : β_0 (le taux « très long ») est une **extrapolation** et saute entre 3 % et 14 %. Dès l'arrivée des 4-5 ans (mars 2025), puis des 8-10 ans (août 2025), β_0 se stabilise autour de 3-4 % et β_1 autour de -1 %. Le modèle n'est fiable que lorsque la courbe a des points longs.

2.7 Taux zéro-coupon, facteurs d'actualisation et taux forward

La courbe observée est une courbe de **taux au pair** (taux de titres avec coupon). Pour actualiser des flux, il faut des **taux zéro-coupon** : le taux d'un placement qui ne verse qu'un seul flux à l'échéance.

Bootstrapping (année par année) : une obligation au pair de maturité n et de coupon c_n vaut 100 :

$$100 = c_n \times 100 \times (DF_1 + \cdots + DF_{n-1}) + (100 + c_n \times 100) \times DF_n$$

donc $DF_n = \dfrac{1 - c_n(DF_1 + \cdots + DF_{n-1})}{1 + c_n}$. On connaît DF_1 (taux à 1 an), on en déduit DF_2, puis DF_3, etc. Le taux zéro-coupon est $z_n = DF_n^{-1/n} - 1$.

Il faut un taux au pair pour **chaque année** de 1 à 10 ans : les maturités 6 et 7 ans (non cotées) sont lues sur la courbe construite. Pour 1 an ou moins, le taux zéro-coupon est égal au taux observé.

Taux forward entre deux maturités $t_1 < t_2$: le taux, fixé aujourd'hui, d'un placement qui commence en t_1 et finit en t_2 : $f = (DF_{t_1}/DF_{t_2})^{1/(t_2 - t_1)} - 1$.

In [17]:
```
GRILLE_ZC = np.array([4/52, 13/52, 26/52, 1, 2, 3, 4, 5, 6, 7, 8, 9,
10])   # maturités de la courbe zéro-coupon

def bootstrap(taux_au_pair):
    # taux_au_pair : fonction qui donne le taux au pair pour une
maturité T
    # Renvoie la courbe zéro-coupon (taux actuariels) sur GRILLE_ZC
    zc = {}
    for T in [4/52, 13/52, 26/52]:
        zc[T] = taux_au_pair(T)                    # ≤ 1 an : zéro-
coupon = taux observé
    somme_df = 0.0
    for n in range(1, 11):
        c = taux_au_pair(n)
        df_n = (1 - c * somme_df) / (1 + c)
        somme_df += df_n
        zc[n] = df_n ** (-1 / n) - 1
    return pd.Series(zc)

def facteur_actualisation(t, zc):
    # DF(t) = 1 / (1 + z(t))^t, avec z interpolé linéairement entre les
```

```
points de la courbe zéro-coupon
    z = np.interp(t, zc.index, zc.values)
    return (1 + z) ** (-t)

def forwards(zc):
    # Taux forward entre deux points consécutifs de la courbe zéro-
coupon
    t = zc.index.values
    df = facteur_actualisation(t, zc)
    f = (df[:-1] / df[1:]) ** (1 / (t[1:] - t[:-1])) - 1
    return pd.Series(f, index=t[1:])

# Courbe zéro-coupon au 12/02/2026 pour chacune des trois méthodes
zc_lin = bootstrap(lambda T: courbe_lineaire(T_ref, y_ref, T))
zc_spl = bootstrap(lambda T: courbe_spline(T_ref, y_ref, T))
zc_ns = bootstrap(lambda T: courbe_ns(params_ref, T)[0])

tableau_zc = pd.DataFrame({
    "taux au pair (%)": [courbe_lineaire(T_ref, y_ref, t) * 100 for t in
GRILLE_ZC],
    "zéro-coupon (%)": zc_lin.values * 100,
    "facteur d'actualisation": facteur_actualisation(GRILLE_ZC, zc_lin),
    "forward depuis le point précédent (%)": [np.nan] +
list(forwards(zc_lin).values * 100),
    "zéro-coupon spline (%)": zc_spl.values * 100,
    "zéro-coupon Nelson-Siegel (%)": zc_ns.values * 100,
}, index=np.round(GRILLE_ZC, 3))
tableau_zc.index.name = "maturité (ans)"
tableau_zc
```

Out[17]:

maturité (ans)	taux au pair (%)	zéro-coupon (%)	facteur d'actualisation	forward depuis le point précédent (%)	zéro-coupon spline (%)	zéro-coupon Nelson-Siegel (%)
0.077	2.059	2.059	0.998	NaN	2.059	3.018
0.250	4.184	4.184	0.990	5.143	4.184	3.289
0.500	4.387	4.387	0.979	4.590	4.387	3.625
1.000	3.560	3.560	0.966	2.739	3.560	4.133
2.000	3.930	3.937	0.926	4.316	3.937	4.686
3.000	5.490	5.586	0.850	8.963	5.586	4.847
4.000	4.920	4.940	0.825	3.025	4.940	4.798
5.000	4.670	4.661	0.796	3.551	4.661	4.645
6.000	4.350	4.306	0.776	2.552	4.337	4.451
7.000	4.030	3.955	0.762	1.873	3.872	4.248
8.000	3.710	3.607	0.753	1.203	3.609	4.054
9.000	3.840	3.766	0.717	5.046	3.768	3.876
10.000	4.150	4.141	0.666	7.578	4.143	3.717

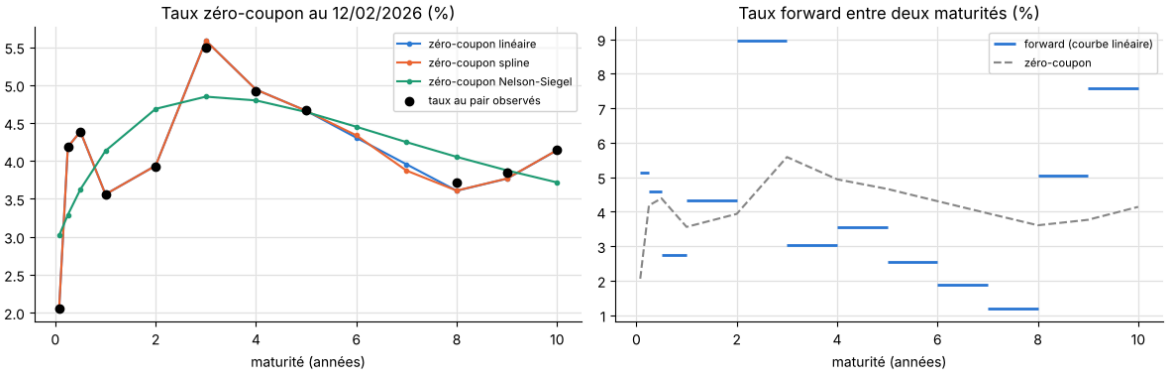
In [18]:

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for zc, nom, couleur in [(zc_lin, "linéaire", BLEU), (zc_spl, "spline",
ORANGE), (zc_ns, "Nelson-Siegel", VERT)]:
    axes[0].plot(zc.index, zc * 100, color=couleur, marker="o",
markersize=3, label="zéro-coupon " + nom)
axes[0].scatter(T_ref, y_ref * 100, color="black", zorder=3, label="taux
au pair observés")
axes[0].set_title("Taux zéro-coupon au 12/02/2026 (%)")
axes[0].set_xlabel("maturité (années)")
axes[0].legend(fontsize=8)
```

```
fwd = forwards(zc_lin)
debuts = zc_lin.index.values[:-1]
axes[1].hlines(fwd * 100, debuts, fwd.index, color=BLEU, linewidth=2,
label="forward (courbe linéaire)")
axes[1].plot(zc_lin.index, zc_lin * 100, color=GRIS, linestyle="--",
label="zéro-coupon")
axes[1].set_title("Taux forward entre deux maturités (%)")
axes[1].set_xlabel("maturité (années)")
axes[1].legend(fontsize=8)
plt.tight_layout()
plt.show()

print("Facteur d'actualisation à 10 ans :",
round(facteur_actualisation(10, zc_lin), 4),
      "-> 100 MRU reçus dans 10 ans valent", round(100 *
facteur_actualisation(10, zc_lin), 1), "MRU aujourd'hui")
```




```
Facteur d'actualisation à 10 ans : 0.6665 -> 100 MRU reçus dans 10 ans val
ent 66.6 MRU aujourd'hui
```

Interprétation :

- Les taux **zéro-coupon** sont proches des taux au pair, et un peu au-dessus quand la courbe monte, un peu en dessous quand elle baisse : c'est normal, car un taux au pair est une moyenne des taux zéro-coupon pondérée par les flux.
- Les **forwards** amplifient les irrégularités de la courbe : quand la courbe zéro-coupon monte fortement (entre 1 et 3 ans), le forward est très élevé ; quand elle baisse (entre 5 et 8 ans), le forward devient faible. Un forward de 1 % ou de 9 % n'est pas une vraie anticipation de marché : c'est la trace d'adjudications isolées.
- Linéaire et spline donnent presque les mêmes zéro-coupons : elles passent par les mêmes points et ne diffèrent que de quelques pb à 6-7 ans (maturités non cotées). **Nelson-Siegel s'écarte nettement** (3,0 % au lieu de 2,1 % à 4 semaines, 4,1 % au lieu de 3,6 % à 1 an, 4,7 % au lieu de 3,9 % à 2 ans) : il lisse les irrégularités au lieu de les reproduire.
- Les facteurs d'actualisation diminuent bien avec la maturité (contrôle de cohérence : pas de taux négatif implicite).

3. Dynamique de la courbe : analyse en composantes principales (ACP)

Idée : les taux de différentes maturités bougent souvent ensemble. L'ACP cherche les quelques « mouvements types » qui expliquent l'essentiel des variations. Sur les grands marchés (États-Unis, zone euro), on trouve presque toujours trois facteurs :

1. **niveau** : toute la courbe monte ou baisse (environ 80-90 % de la variance) ;
2. **pente** : les taux courts et longs bougent en sens inverse ;
3. **courbure** : le milieu de la courbe bouge différemment des deux extrémités.

Méthode : on calcule la matrice de variance-covariance des variations hebdomadaires, puis ses **vecteurs propres** (les mouvements types, ou « loadings ») et ses **valeurs propres** (la variance expliquée par chacun).

3.1 ACP sur les variations hebdomadaires observées

Comme décidé, on travaille sur les 4 bons (4S, 13S, 26S, 52S), disponibles ensemble depuis le 16/11/2023. Les obligations sont exclues : leurs taux ne changent presque jamais d'une semaine à l'autre (variation nulle 95 % du temps ou plus).

In [19]:
```
def acp(variations):
    # ACP sur une matrice de variations (une colonne par maturité)
    valeurs_propres, vecteurs_propres = np.linalg.eigh(variations.cov())
    ordre = np.argsort(valeurs_propres)[::-1]                # du plus
grand au plus petit
    valeurs_propres = valeurs_propres[ordre]
    vecteurs_propres = vecteurs_propres[:, ordre]
    # Convention de signe : chaque composante a une somme de loadings
positive (lecture plus facile)
    vecteurs_propres = vecteurs_propres *
np.sign(vecteurs_propres.sum(axis=0))
    noms_cp = [f"CP{i + 1}" for i in range(len(ordre))]
    part_variance = pd.Series(valeurs_propres / valeurs_propres.sum() *
100, index=noms_cp)
    loadings = pd.DataFrame(vecteurs_propres, index=variations.columns,
columns=noms_cp)
    return part_variance, loadings

BONS = ["4S", "13S", "26S", "52S"]
variations_bons = (taux[BONS].dropna().diff().dropna()) * 10000     # en
points de base

print(len(variations_bons), "variations hebdomadaires, du",
variations_bons.index.min().date(),
      "au", variations_bons.index.max().date())
print("\nPart des semaines sans variation :")
print((variations_bons == 0).mean().round(2).to_dict())
print("\nCorrélation des variations hebdomadaires :")
variations_bons.corr()
```

```
117 variations hebdomadaires, du 2023-11-23 au 2026-02-12

Part des semaines sans variation :
{'4S': 0.13, '13S': 0.42, '26S': 0.41, '52S': 0.9}

Corrélation des variations hebdomadaires :
```

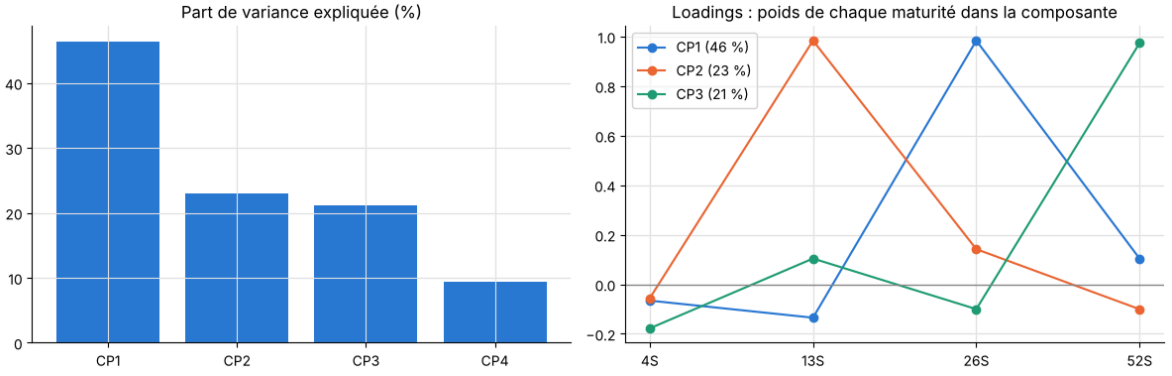
Out[19]:

	4S	13S	26S	52S
4S	1.000	-0.042	-0.107	-0.152
13S	-0.042	1.000	-0.097	-0.030
26S	-0.107	-0.097	1.000	0.078
52S	-0.152	-0.030	0.078	1.000

In [20]:

```
part_bons, loadings_bons = acp(variations_bons)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].bar(part_bons.index, part_bons, color=BLEU)
axes[0].set_title("Part de variance expliquée (%)")
for cp, couleur in zip(["CP1", "CP2", "CP3"], [BLEU, ORANGE, VERT]):
    axes[1].plot(BONS, loadings_bons[cp], marker="o", color=couleur,
label=f"{cp} ({part_bons[cp]:.0f} %)")
axes[1].axhline(0, color=GRIS, linewidth=1)
axes[1].set_title("Loadings : poids de chaque maturité dans la
composante")
axes[1].legend()
plt.tight_layout()
plt.show()
loadings_bons.round(2)
```



Out[20]:

	CP1	CP2	CP3	CP4
4S	-0.070	-0.060	-0.180	0.980
13S	-0.130	0.980	0.100	0.070
26S	0.980	0.140	-0.100	0.060
52S	0.100	-0.100	0.970	0.180

Interprétation (résultat important) :

- Les variations hebdomadaires des 4 bons sont **presque indépendantes** (corrélations entre -0,15 et +0,08), alors que sur un marché développé elles seraient corrélées à plus de 0,7.
- Par conséquent, l'ACP **ne trouve pas de facteur niveau** : chaque composante correspond presque à **une seule maturité** (la première au 26S, la deuxième au 13S, etc.). La première composante n'explique qu'environ 46 % de la variance, contre 80-90 % pour un facteur niveau sur un grand marché.

- **Pourquoi ?** Les bons ne sont pas adjugés la même semaine : quand le taux du 26S change, celui du 52S reste figé parce qu'il n'y a pas eu d'adjudication à 52 semaines. Et le résultat de chaque adjudication dépend de la demande des banques ce jour-là. On mesure surtout **le bruit propre à chaque adjudication**, pas un mouvement commun de la courbe.

C'est un résultat en soi : **sur le marché mauritanien, on ne peut pas lire niveau / pente / courbure dans les données brutes hebdomadaires.**

3.2 ACP sur la courbe lissée par Nelson-Siegel

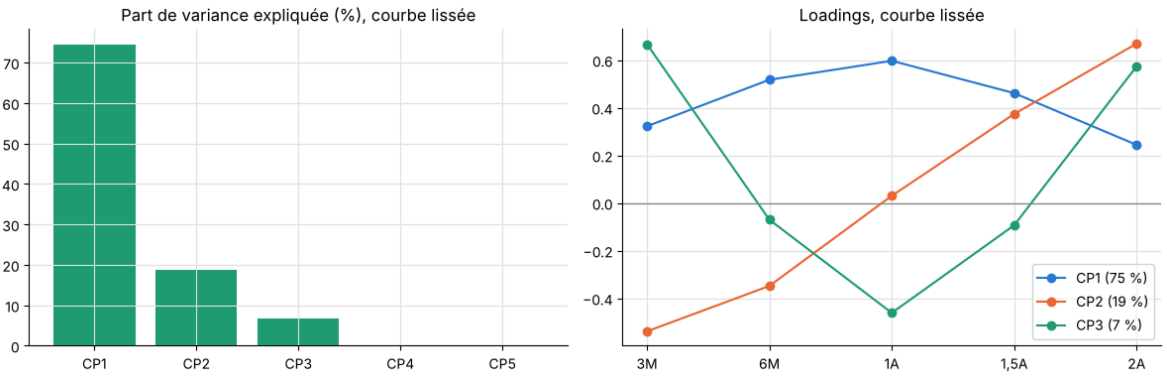
Pour retrouver des facteurs de courbe, on applique l'ACP à la courbe **lissée** : chaque semaine, on lit la courbe Nelson-Siegel (λ fixé) à des maturités fixes (3 mois, 6 mois, 1, 1,5 et 2 ans, toujours couvertes par des titres sur les 105 semaines). Le lissage relie chaque maturité à ses voisines : un mouvement d'un point se diffuse sur toute la courbe.

Attention : avec 3 facteurs Nelson-Siegel, la courbe lissée n'a, par construction, que 3 façons de bouger. Les 3 premières composantes expliqueront donc 100 % de la variance : ce chiffre ne prouve rien. Ce qui est intéressant, c'est **l'ordre et la forme** des composantes.

In [21]:
```
MATURITES_FIXES = np.array([0.25, 0.5, 1, 1.5, 2])
courbe_lissee = pd.DataFrame(
    [facteurs_ns(MATURITES_FIXES, LAMBDA_FIXE) @ params_fixe.loc[d,
["beta0", "beta1", "beta2"]].values
     for d in params_fixe.index],
    index=params_fixe.index, columns=["3M", "6M", "1A", "1,5A", "2A"])
variations_lissees = courbe_lissee.diff().dropna() * 10000

part_ns, loadings_ns = acp(variations_lissees)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].bar(part_ns.index, part_ns, color=VERT)
axes[0].set_title("Part de variance expliquée (%), courbe lissée")
for cp, couleur in zip(["CP1", "CP2", "CP3"], [BLEU, ORANGE, VERT]):
    axes[1].plot(courbe_lissee.columns, loadings_ns[cp], marker="o",
color=couleur, label=f"{cp} ({part_ns[cp]:.0f} %)")
axes[1].axhline(0, color=GRIS, linewidth=1)
axes[1].set_title("Loadings, courbe lissée")
axes[1].legend()
plt.tight_layout()
plt.show()
print("Part de variance expliquée (%) :", part_ns.round(1).to_dict())
loadings_ns.round(2)
```




```
Part de variance expliquée (%) : {'CP1': 74.5, 'CP2': 18.8, 'CP3': 6.6, 'C
P4': 0.0, 'CP5': 0.0}
```

Out[21]:

	CP1	CP2	CP3	CP4	CP5
3M	0.320	-0.540	0.670	-0.340	-0.210
6M	0.520	-0.350	-0.070	0.730	0.280
1A	0.600	0.030	-0.460	-0.580	0.310
1,5A	0.460	0.370	-0.090	0.150	-0.790
2A	0.240	0.670	0.570	0.050	0.400

Interprétation : on lit chaque composante d'après le signe de ses loadings :

- tous les loadings du même signe = **niveau** (toute la courbe bouge dans le même sens) ;
- loadings qui changent de signe une fois = **pente** (le court et le long bougent en sens inverse) ;
- loadings qui changent de signe deux fois (forme en U ou en cloche) = **courbure**.

Le texte ci-dessous est écrit à partir des résultats affichés.

- **CP1 (environ 75 % de la variance) = niveau** : tous les loadings sont positifs, toute la courbe courte monte ou baisse ensemble (un peu plus vers 6 mois-1 an).
- **CP2 (environ 19 %) = pente** : loadings négatifs à 3-6 mois et positifs à 1,5-2 ans ; les taux courts et longs bougent en sens inverse.
- **CP3 (environ 7 %) = courbure** : loadings positifs aux extrémités (3 mois, 2 ans) et négatifs au milieu (1 an) ; le milieu de la courbe bouge à l'inverse des bords.

On retrouve donc bien la hiérarchie classique niveau > pente > courbure, mais **seulement sur la courbe lissée**. La comparaison avec 3.1 montre que ces facteurs viennent en partie du lissage : dans les données brutes, les maturités ne bougent pas ensemble. La part du niveau (75 %) est aussi plus faible que sur les grands marchés (80-90 %).

3.3 Interprétation économique pour le marché mauritanien

- **Le taux directeur est le vrai facteur « niveau ».** Sur la période, les grandes tendances des taux (hausse en 2023-2024, baisse en 2025) suivent le cycle

de la BCM. Mais elles se transmettent **lentement et adjudication par adjudication**, pas chaque semaine : c'est pour cela que le niveau ressort dans les tendances et pas dans les variations hebdomadaires.

- **La liquidité bancaire joue sur la pente.** Quand les banques ont beaucoup de liquidité (taux de dépôt à 2-3 %, titres sous le taux directeur), elles se placent d'abord sur les maturités courtes : les taux courts baissent plus que les longs.
- **La courbure reflète la structure de l'offre du Trésor.** Le Trésor choisit quelles maturités il émet et quand. Une émission ponctuelle à 3 ans à un taux élevé crée une bosse que personne n'arbitre, faute de marché secondaire.
- **Pour un gérant ou un trésorier de banque**, cela veut dire que le risque principal sur ce marché n'est pas un déplacement parallèle quotidien de la courbe, mais un **risque d'adjudication** (le taux de la prochaine émission) et un **risque de liquidité** (impossibilité de revendre avant l'échéance).

4. Valorisation et risque de taux

4.1 Le portefeuille de titres publics

On valorise un portefeuille **hypothétique** composé de titres réellement émis selon le fichier : la date d'émission est la semaine où le taux d'adjudication apparaît, et le coupon est ce taux d'adjudication (le titre est supposé émis au pair). Le **nominal** de chaque ligne (100 millions MRU) est une hypothèse de travail.

Conventions simples : coupon annuel versé à la date anniversaire de l'émission, temps compté en jours / 365, valorisation au 12/02/2026 sur la courbe zéro-coupon **linéaire** (méthode de la BCM).

In [22]:
```
DATE_VALO = DATE_REF
NOMINAL = 100   # millions de MRU par ligne (hypothèse)

def ligne_titre(nom, type_titre, maturite_col, date_emission,
duree_ans):
    # Construit la description d'un titre à partir du taux
d'adjudication lu dans le fichier
    date_emission = pd.Timestamp(date_emission)
    coupon = taux_brut.loc[date_emission, maturite_col] / 100
    if type_titre == "Bon":
        echeance = date_emission + pd.Timedelta(days=364)
    else:
        echeance = date_emission + pd.DateOffset(years=duree_ans)
    return {"titre": nom, "type": type_titre, "émission": date_emission,
"échéance": echeance,
            "coupon": coupon, "durée (ans)": duree_ans}

portefeuille = pd.DataFrame([
    ligne_titre("Bon 52S déc. 2025", "Bon", "52S", "2025-12-04", 1),
    ligne_titre("OT 2 ans oct. 2024", "OT", "2A", "2024-10-17", 2),
    ligne_titre("OT 3 ans août 2024", "OT", "3A", "2024-08-29", 3),
    ligne_titre("OT 3 ans mars 2025", "OT", "3A", "2025-03-13", 3),
    ligne_titre("OT 5 ans mars 2025", "OT", "5A", "2025-03-13", 5),
    ligne_titre("OT 10 ans août 2025", "OT", "10A", "2025-08-28", 10),
]).set_index("titre")
```

```
portefeuille["vie résiduelle (ans)"] = (portefeuille["échéance"] -
DATE_VALO).dt.days / 365
portefeuille
```

Out[22]:

titre	type	émission	échéance	coupon	durée (ans)	vie résiduelle (ans)
Bon 52S déc. 2025	Bon	2025-12-04	2026-12-03	0.042	1	0.805
OT 2 ans oct. 2024	OT	2024-10-17	2026-10-17	0.060	2	0.677
OT 3 ans août 2024	OT	2024-08-29	2027-08-29	0.070	3	1.542
OT 3 ans mars 2025	OT	2025-03-13	2028-03-13	0.044	3	2.082
OT 5 ans mars 2025	OT	2025-03-13	2030-03-13	0.047	5	4.082
OT 10 ans août 2025	OT	2025-08-28	2035-08-28	0.042	10	9.545

Le bon est un titre sans coupon : il rembourse 100 à l'échéance (son « coupon » affiché est son taux d'émission, il ne sert pas au calcul des flux).

In [23]:

```
def flux_futurs(titre, date_valo=DATE_VALO):
    # Dates (en années à partir d'aujourd'hui) et montants des flux
restants, pour 100 de nominal
    if titre["type"] == "Bon":
        dates = [titre["échéance"]]
        montants = [100.0]
    else:
        dates = [titre["émission"] + pd.DateOffset(years=k) for k in
range(1, titre["durée (ans)"] + 1)]
        montants = [100 * titre["coupon"]] * len(dates)
        montants[-1] += 100                                    #
remboursement du nominal à l'échéance
    t = np.array([(d - date_valo).days / 365 for d in dates])
    montants = np.array(montants)
    return t[t > 0], montants[t > 0]

def coupon_couru(titre, date_valo=DATE_VALO):
    # Part du coupon déjà « gagnée » depuis la dernière date de coupon
(0 pour un bon)
    if titre["type"] == "Bon":
        return 0.0
    dernier = titre["émission"]
    while dernier + pd.DateOffset(years=1) <= date_valo:
        dernier = dernier + pd.DateOffset(years=1)
    return 100 * titre["coupon"] * (date_valo - dernier).days / 365

def prix(titre, zc):
    # Prix plein (coupon couru inclus) = somme des flux actualisés avec
la courbe zéro-coupon
    t, montants = flux_futurs(titre)
    return np.sum(montants * facteur_actualisation(t, zc))
```

4.2 Prix, taux actuariel, duration, convexité et DV01

- **Taux actuariel (YTM)** : le taux unique y qui, utilisé pour actualiser tous les flux, redonne le prix.
- **Duration de Macaulay** : maturité moyenne des flux, pondérée par leur valeur actualisée : $D = \sum t_i \cdot \frac{CF_i/(1+y)^{t_i}}{P}$.
- **Duration modifiée** : $D_{mod} = D/(1+y)$. Si le taux monte de 1 %, le prix baisse d'environ D_{mod} %.

- **Convexité** : $C = \frac{1}{P}\sum \frac{t_i(t_i+1)\,CF_i}{(1+y)^{t_i+2}}$. Elle corrige la duration pour les grands chocs : $\frac{\Delta P}{P} \approx -D_{mod}\Delta y + \frac{1}{2}C\,(\Delta y)^2$.
- **DV01** : perte en valeur pour une hausse de 1 pb **de toute la courbe zéro-coupon**. On la calcule directement en revalorisant le titre (courbe +1 pb et -1 pb), sans passer par la formule de duration.

In [24]:
```
def taux_actuariel(titre, prix_plein):
    # Résout : somme des flux actualisés au taux y = prix plein
    t, montants = flux_futurs(titre)
    ecart = lambda y: np.sum(montants / (1 + y) ** t) - prix_plein
    return brentq(ecart, -0.5, 1.0)

def indicateurs_titre(titre, zc):
    t, montants = flux_futurs(titre)
    p = prix(titre, zc)
    y = taux_actuariel(titre, p)
    valeurs_actualisees = montants / (1 + y) ** t
    duration = np.sum(t * valeurs_actualisees) / p
    convexite = np.sum(t * (t + 1) * montants / (1 + y) ** (t + 2)) / p
    dv01 = (prix(titre, zc - 0.0001) - prix(titre, zc + 0.0001)) / 2
# pour 100 de nominal
    return {"prix plein": p, "coupon couru": coupon_couru(titre), "prix
pied de coupon": p - coupon_couru(titre),
            "taux actuariel (%)": y * 100, "duration Macaulay":
duration, "duration modifiée": duration / (1 + y),
            "convexité": convexite, "DV01 (pour 100)": dv01}

risque = pd.DataFrame({nom: indicateurs_titre(titre, zc_lin) for nom,
titre in portefeuille.iterrows()}).T
risque["valeur (M MRU)"] = risque["prix plein"] * NOMINAL / 100
risque["DV01 (MRU)"] = risque["DV01 (pour 100)"] * NOMINAL / 100 * 1e6
# en MRU, pour la ligne de 100 M
risque.drop(columns="DV01 (pour 100)")
```

Out[24]:

	prix plein	coupon couru	prix pied de coupon	taux actuariel (%)	duration Macaulay	duration modifiée	convexité	valeur (M MRU)	DV01 (MRU)
Bon 52S déc. 2025	96.979	0.000	96.979	3.882	0.805	0.775	1.348	96.979	7519.570
OT 2 ans oct. 2024	103.160	1.940	101.220	4.095	0.677	0.650	1.047	103.160	6706.357
OT 3 ans août 2024	107.932	3.207	104.725	3.778	1.479	1.425	3.459	107.932	15381.965
OT 3 ans mars 2025	104.788	4.078	100.710	4.059	1.957	1.881	5.524	104.788	19705.687
OT 5 ans mars 2025	103.456	4.299	99.157	4.897	3.653	3.482	16.468	103.456	35998.647
OT 10 ans août 2025	102.925	1.910	101.015	4.015	7.939	7.633	72.900	102.925	78764.052

In [25]:
```
# Indicateurs du portefeuille : sommes (valeur, DV01) et moyennes
pondérées par la valeur (duration, convexité)
valeur_ptf = risque["valeur (M MRU)"].sum()
poids = risque["valeur (M MRU)"] / valeur_ptf
duration_ptf = (poids * risque["duration modifiée"]).sum()
convexite_ptf = (poids * risque["convexité"]).sum()
dv01_ptf = risque["DV01 (MRU)"].sum() / 1e6      # en millions de MRU

print(f"Valeur du portefeuille   : {valeur_ptf:.2f} M MRU (nominal
{NOMINAL * len(portefeuille)} M MRU)")
```

```
print(f"Duration modifiée        : {duration_ptf:.2f}")
print(f"Convexité                : {convexite_ptf:.2f}")
print(f"DV01                     : {dv01_ptf * 1e6:,.0f} MRU par point
de base".replace(",", " "))
print(f"Contrôle : DV01 ≈ duration × valeur × 0.0001 = {duration_ptf *
valeur_ptf * 0.0001 * 1e6:,.0f} MRU".replace(",", " "))
```

```
Valeur du portefeuille   : 619.24 M MRU (nominal 600 M MRU)
Duration modifiée        : 2.65
Convexité                : 16.79
DV01                     : 164 076 MRU par point de base
Contrôle : DV01 ≈ duration × valeur × 0.0001 = 163 902 MRU
```

Interprétation :

- **Contrôle de cohérence :** les obligations récentes dont le coupon est proche des taux actuels valent entre 99 et 101 (prix pied de coupon). C'est attendu : la courbe est construite à partir de ces mêmes taux d'adjudication.
- Les titres émis **quand les taux étaient hauts** (OT 2 ans à 6 %, OT 3 ans à 7,01 % de 2024) valent **plus que 100** : leur coupon est supérieur aux taux actuels (2-5 %). Le détenteur a gagné de l'argent avec la baisse des taux de la BCM.
- La **duration** augmente avec la maturité : l'OT 10 ans (duration modifiée de 7,6) porte à elle seule près de la moitié du DV01 du portefeuille. Le portefeuille a une duration modifiée de 2,65 : une hausse de 1 pb de toute la courbe lui fait perdre environ 164 000 MRU.
- Le DV01 calculé par revalorisation est très proche de « duration × valeur × 0,0001 » : les deux approches sont cohérentes.

4.3 Key rate durations (KRD)

La duration suppose que toute la courbe bouge du même montant. Les **key rate durations** mesurent la sensibilité à **chaque point** de la courbe séparément : on monte de 1 pb un seul taux zéro-coupon (par exemple le 5 ans), on laisse les autres inchangés, et on mesure la variation de prix. Comme on interpole linéairement entre les points, ce choc se diffuse en « triangle » jusqu'aux points voisins.

Contrôle : la **somme des KRD** doit redonner la duration effective (choc parallèle).

In [26]:
```
def key_rate_durations(titre, zc):
    p = prix(titre, zc)
    krd = {}
    for T in zc.index:
        zc_choque = zc.copy()
        zc_choque[T] += 0.0001
        krd[T] = -(prix(titre, zc_choque) - p) / p / 0.0001
    return pd.Series(krd)

krd = pd.DataFrame({nom: key_rate_durations(titre, zc_lin) for nom,
titre in portefeuille.iterrows()})
krd.index = [f"{t:.2f} an" if t < 1 else f"{t:.0f} ans" for t in
krd.index]

# KRD du portefeuille = moyenne des KRD pondérée par la valeur
krd["Portefeuille"] = krd[portefeuille.index].mul(poids,
```

```
axis=1).sum(axis=1)

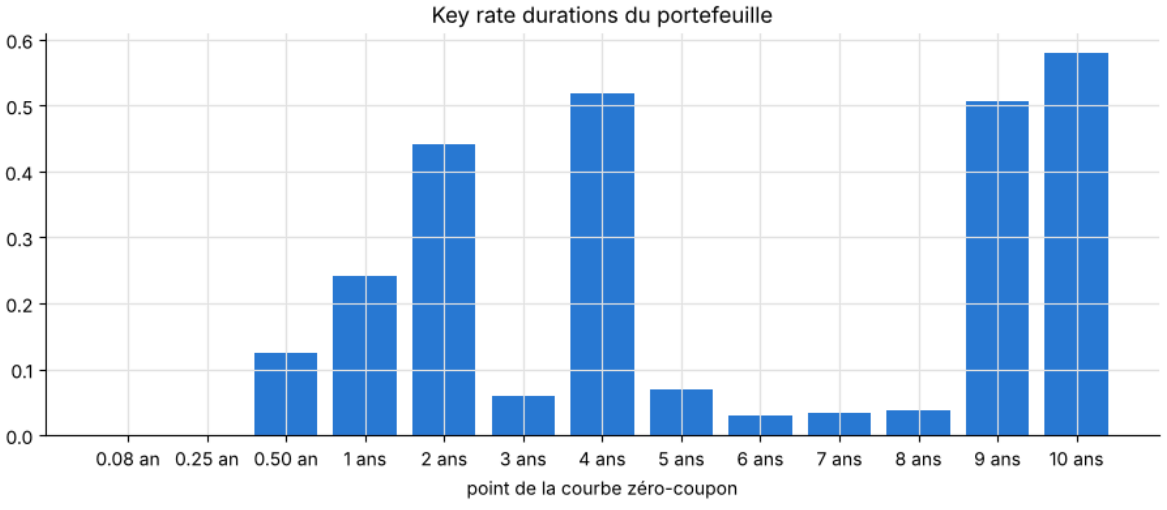
duration_effective = risque["DV01 (pour 100)"] / risque["prix plein"] *
10000
controle_krd = pd.DataFrame({"somme des KRD":
krd[portefeuille.index].sum(),
                             "duration effective": duration_effective})
print(controle_krd.round(3))
krd.round(3) + 0.0     # + 0.0 évite l'affichage « -0.000 »
```

```
                    somme des KRD  duration effective
Bon 52S déc. 2025           0.775               0.775
OT 2 ans oct. 2024          0.650               0.650
OT 3 ans août 2024          1.425               1.425
OT 3 ans mars 2025          1.880               1.881
OT 5 ans mars 2025          3.479               3.480
OT 10 ans août 2025         7.651               7.653
```

Out[26]:

	Bon 52S déc. 2025	OT 2 ans oct. 2024	OT 3 ans août 2024	OT 3 ans mars 2025	OT 5 ans mars 2025	OT 10 ans août 2025	Portefeuille
0.08 an	0.000	0.000	0.000	0.003	0.003	0.000	0.001
0.25 an	0.000	0.000	0.000	0.000	0.000	0.000	0.000
0.50 an	0.302	0.420	0.030	0.000	0.000	0.019	0.126
1 ans	0.474	0.230	0.640	0.039	0.042	0.028	0.242
2 ans	0.000	0.000	0.755	1.687	0.080	0.070	0.442
3 ans	0.000	0.000	0.000	0.151	0.109	0.099	0.060
4 ans	0.000	0.000	0.000	0.000	2.979	0.126	0.519
5 ans	0.000	0.000	0.000	0.000	0.266	0.153	0.070
6 ans	0.000	0.000	0.000	0.000	0.000	0.180	0.030
7 ans	0.000	0.000	0.000	0.000	0.000	0.207	0.034
8 ans	0.000	0.000	0.000	0.000	0.000	0.232	0.039
9 ans	0.000	0.000	0.000	0.000	0.000	3.046	0.506
10 ans	0.000	0.000	0.000	0.000	0.000	3.492	0.580

In [27]:
```
fig, ax = plt.subplots(figsize=(11, 4))
ax.bar(krd.index, krd["Portefeuille"], color=BLEU)
ax.set_title("Key rate durations du portefeuille")
ax.set_xlabel("point de la courbe zéro-coupon")
plt.show()
```



Interprétation :

- La somme des KRD est égale à la duration effective : le découpage est cohérent.
- Chaque titre n'est sensible qu'aux points de la courbe **autour de ses flux** : le bon 52S et l'OT 2 ans dépendent surtout des taux à moins de 1 an, l'OT 10 ans surtout du point 9-10 ans.
- Pour le portefeuille, le risque est concentré là où tombent les remboursements : **2 ans** (OT 3 ans de mars 2025, qui échoit dans 2,1 ans), **4 ans** (OT 5 ans, 4,1 ans restants) et **9-10 ans** (OT 10 ans). Les points 6-7 ans, non cotés, reçoivent aussi un peu de risque (coupons de l'OT 10 ans) : leur taux est une interpolation, ce qui est une source de risque de modèle.
- Utilité pratique : pour couvrir le portefeuille, il faudrait vendre des titres **sur ces maturités-là**, pas seulement réduire la duration globale.

4.4 Scénarios de choc

On applique trois chocs à la courbe zéro-coupon et on revalorise chaque titre (revalorisation complète, pas d'approximation) :

- **Hausse parallèle de +100 pb** : toute la courbe monte de 1 %.
- **Pentification** : les taux courts baissent, les longs montent : -50 pb à 0 an, +50 pb à 10 ans, linéaire entre les deux (pivot à 5 ans).
- **Aplatissement** : l'inverse, +50 pb à 0 an et -50 pb à 10 ans.

In [28]:
```
def choc_parallele(T):
    return 0.01 * np.ones_like(T)

def choc_pentification(T):
    return (-50 + 100 * T / 10) / 10000

def choc_aplatissement(T):
    return -choc_pentification(T)

SCENARIOS = {"hausse parallèle +100 pb": choc_parallele,
             "pentification": choc_pentification,
             "aplatissement": choc_aplatissement}

impacts = {}
for nom_scenario, choc in SCENARIOS.items():
    zc_choque = zc_lin + choc(zc_lin.index.values)
    impacts[nom_scenario] = [(prix(titre, zc_choque) - prix(titre,
zc_lin)) * NOMINAL / 100
                             for _, titre in portefeuille.iterrows()]

impacts = pd.DataFrame(impacts, index=portefeuille.index)   # en
millions de MRU
impacts.loc["Portefeuille"] = impacts.sum()
impacts.loc["Portefeuille (% de la valeur)"] =
impacts.loc["Portefeuille"] / valeur_ptf * 100
impacts
```

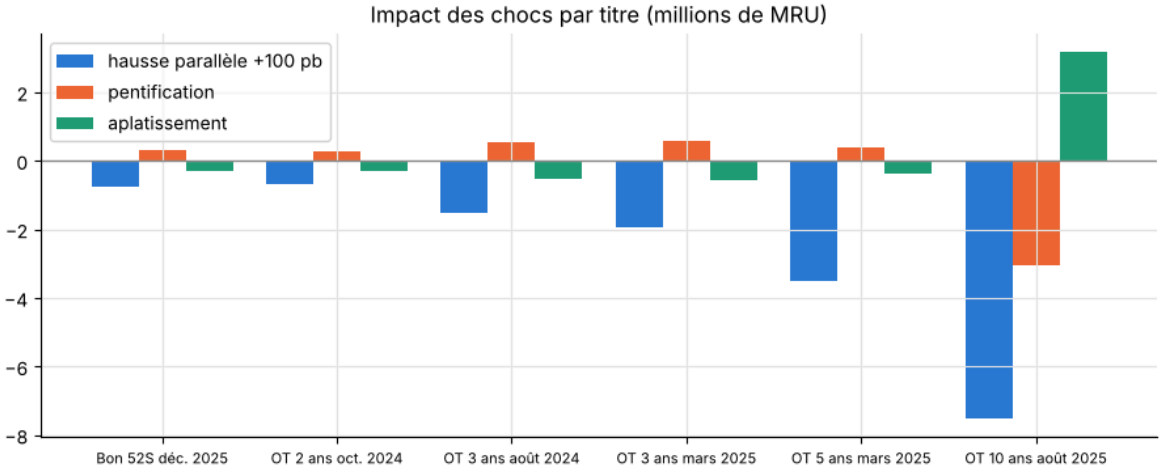
Out[28]:

titre	hausse parallèle +100 pb	pentification	aplatissement
Bon 52S déc. 2025	-0.745	0.317	-0.314
OT 2 ans oct. 2024	-0.665	0.291	-0.289
OT 3 ans août 2024	-1.520	0.538	-0.533
OT 3 ans mars 2025	-1.942	0.583	-0.578
OT 5 ans mars 2025	-3.516	0.376	-0.374
OT 10 ans août 2025	-7.513	-3.039	3.182
Portefeuille	-15.902	-0.935	1.094
Portefeuille (% de la valeur)	-2.568	-0.151	0.177

In [29]:
```
# Contrôle de la hausse parallèle avec l'approximation duration + convexité
dy = 0.01
approx_duration = -duration_ptf * dy * 100
approx_dur_conv = (-duration_ptf * dy + 0.5 * convexite_ptf * dy ** 2) * 100
print(f"Revalorisation complète : {impacts.loc['Portefeuille (% de la valeur)', 'hausse parallèle +100 pb']:.3f} %")
print(f"Duration seule          : {approx_duration:.3f} %")
print(f"Duration + convexité    : {approx_dur_conv:.3f} %")

fig, ax = plt.subplots(figsize=(11, 4))
x = np.arange(len(portefeuille))
for i, (nom_scenario, couleur) in enumerate(zip(SCENARIOS, [BLEU, ORANGE, VERT])):
    ax.bar(x + (i - 1) * 0.27, impacts.loc[portefeuille.index, nom_scenario], width=0.27, color=couleur, label=nom_scenario)
ax.axhline(0, color=GRIS, linewidth=1)
ax.set_xticks(x)
ax.set_xticklabels(portefeuille.index, fontsize=8)
ax.set_title("Impact des chocs par titre (millions de MRU)")
ax.legend()
plt.show()
```

```
Revalorisation complète : -2.568 %
Duration seule          : -2.647 %
Duration + convexité    : -2.563 %
```



Interprétation :

- **Hausse parallèle de +100 pb** : c'est le scénario le plus coûteux. La perte est proche de « duration × 1 % » de la valeur, et l'approximation **duration +**

convexité est très proche de la revalorisation complète (la convexité réduit légèrement la perte).

- **Pentification** : les titres longs (OT 10 ans, OT 5 ans) perdent, les titres courts gagnent un peu. Le portefeuille perd globalement parce que son risque est concentré sur des maturités au-delà du pivot de 5 ans (l'OT 10 ans).
- **Aplatissement** : c'est l'inverse, le portefeuille gagne.
- Leçon : deux portefeuilles de même duration peuvent réagir très différemment à un changement de **pente**. C'est exactement ce que montrent les key rate durations.

5. Modèle de Vasicek et simulation Monte Carlo

5.1 Le modèle

Le modèle de Vasicek décrit le taux court r_t par :

$$dr_t = \kappa(\theta - r_t)\,dt + \sigma\,dW_t$$

- θ : niveau moyen de long terme vers lequel le taux revient ;
- κ : vitesse de retour à la moyenne (plus κ est grand, plus le retour est rapide) ;
- σ : volatilité du taux court ;
- dW_t : choc aléatoire (mouvement brownien).

Taux court utilisé : le bon à 4 semaines, le titre le plus court du fichier (converti en taux continu : $r = \ln(1+y)$, la convention du modèle).

5.2 Calibration par régression

Sur un pas de temps fixe Δt (une semaine = 1/52 an), le modèle de Vasicek s'écrit exactement comme une régression linéaire :

$$r_{t+\Delta t} = a + b\,r_t + \varepsilon_t \quad \text{avec} \quad b = e^{-\kappa \Delta t}, \quad a = \theta(1-b)$$

On estime a et b par moindres carrés, puis : $\kappa = -\ln(b)/\Delta t$, $\theta = a/(1-b)$, et $\sigma = \text{écart-type}(\varepsilon) \times \sqrt{2\kappa/(1-b^2)}$.

On ne garde que les couples de semaines **consécutives** (écart de 6 à 8 jours) pour respecter le pas fixe : les trous de plusieurs semaines sont exclus.

In [30]:
```
taux_court = np.log(1 + taux["4S"].dropna())          # taux continu, en décimal
ecart = taux_court.index.to_series().diff().dt.days
consecutives = ecart.between(6, 8)

r_avant = taux_court.shift(1)[consecutives]
r_apres = taux_court[consecutives]

regression = sm.OLS(r_apres, sm.add_constant(r_avant)).fit()
a, b = regression.params
DT = 1 / 52
```

```python
KAPPA = -np.log(b) / DT
THETA = a / (1 - b)
SIGMA = regression.resid.std(ddof=2) * np.sqrt(2 * KAPPA / (1 - b ** 2))
R0 = taux_court.iloc[-1]

print(regression.summary().tables[1])
print(f"\nNombre de couples de semaines : {len(r_apres)}")
print(f"kappa = {KAPPA:.3f}  (demi-vie du retour à la moyenne :
{np.log(2) / KAPPA * 52:.0f} semaines)")
print(f"theta = {THETA * 100:.2f} %")
print(f"sigma = {SIGMA * 100:.2f} %")
print(f"r0    = {R0 * 100:.2f} % (4 semaines au 12/02/2026)")
```

```
==============================================================================
====
                 coef    std err          t      P>|t|      [0.025      0.
975]
------------------------------------------------------------------------------
----
const          0.0012      0.001      1.304      0.194      -0.001
0.003
4S             0.9750      0.017     56.093      0.000       0.941
1.009
==============================================================================
====

Nombre de couples de semaines : 168
kappa = 1.318  (demi-vie du retour à la moyenne : 27 semaines)
theta = 4.92 %
sigma = 3.31 %
r0    = 2.04 % (4 semaines au 12/02/2026)
```

Interprétation :

- $b = 0{,}975$ donne un retour à la moyenne avec une **demi-vie d'environ 27 semaines** (6 mois). **Mais l'intervalle de confiance de b (0,941 à 1,009) contient 1** : statistiquement, on ne peut pas exclure que le taux court soit une marche aléatoire, sans retour à la moyenne. Avec 168 semaines et un seul cycle de hausse puis de baisse, κ et θ sont donc **estimés avec beaucoup d'incertitude**. On garde les valeurs estimées, mais il faut les lire comme des ordres de grandeur.
- θ (environ 4,9 %) est le niveau moyen du 4 semaines sur la période 2022-2026, qui couvre à la fois le resserrement (taux jusqu'à 8 %) et l'assouplissement (2 % aujourd'hui). Le modèle s'attend donc à ce que le taux court **remonte** depuis 2 %. C'est une limite : la calibration historique ne sait pas que la BCM a baissé son taux directeur de 8 % à 6 %.
- σ (environ 3,3 % par an) est **élevé** : sur un grand marché, la volatilité du taux court est plutôt de 1 %. Ici, elle mesure surtout les **sauts d'une adjudication à l'autre**, pas une volatilité de marché au sens classique.

5.3 Simulation Monte Carlo des trajectoires

On simule le taux court semaine par semaine avec la **solution exacte** du modèle (pas d'erreur de discrétisation) :

$$r_{t+\Delta t} = \theta + (r_t - \theta)e^{-\kappa \Delta t} + \sigma \sqrt{\frac{1 - e^{-2\kappa \Delta t}}{2\kappa}}\, Z, \qquad Z \sim \mathcal{N}(0,1)$$

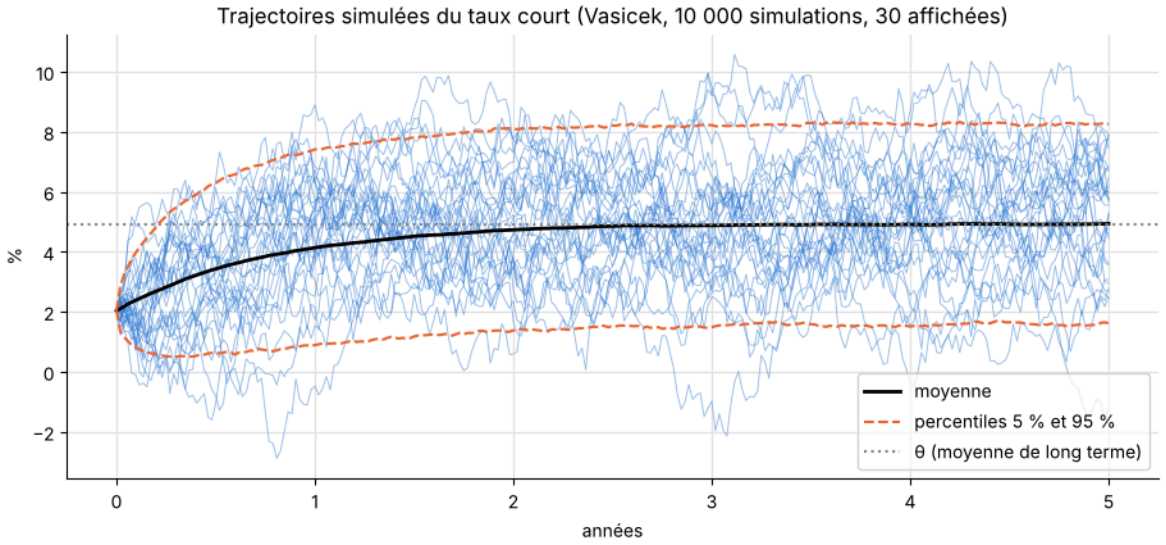
In [31]:

```
def simuler_vasicek(r0, kappa, theta, sigma, horizon_ans,
nb_trajectoires, graine=42):
    # Renvoie une matrice (nb_trajectoires x nb_pas+1) de taux courts
simulés, pas hebdomadaire
    generateur = np.random.default_rng(graine)
    nb_pas = int(round(horizon_ans / DT))
    ecart_type_pas = sigma * np.sqrt((1 - np.exp(-2 * kappa * DT)) / (2
* kappa))
    r = np.zeros((nb_trajectoires, nb_pas + 1))
    r[:, 0] = r0
    for k in range(nb_pas):
        Z = generateur.standard_normal(nb_trajectoires)
        r[:, k + 1] = theta + (r[:, k] - theta) * np.exp(-kappa * DT) +
ecart_type_pas * Z
    return r

trajectoires = simuler_vasicek(R0, KAPPA, THETA, SIGMA, horizon_ans=5,
nb_trajectoires=10000)
temps = np.arange(trajectoires.shape[1]) * DT

fig, ax = plt.subplots(figsize=(11, 4.5))
ax.plot(temps, trajectoires[:30].T * 100, color=BLEU, linewidth=0.6,
alpha=0.5)
ax.plot(temps, trajectoires.mean(axis=0) * 100, color="black",
linewidth=2, label="moyenne")
ax.plot(temps, np.percentile(trajectoires, 5, axis=0) * 100,
color=ORANGE, linestyle="--", label="percentiles 5 % et 95 %")
ax.plot(temps, np.percentile(trajectoires, 95, axis=0) * 100,
color=ORANGE, linestyle="--")
ax.axhline(THETA * 100, color=GRIS, linestyle=":", label="θ (moyenne de
long terme)")
ax.set_title("Trajectoires simulées du taux court (Vasicek, 10 000
simulations, 30 affichées)")
ax.set_xlabel("années")
ax.set_ylabel("%")
ax.legend()
plt.show()

print(f"Part des trajectoires avec un taux négatif à un moment :
{(trajectoires.min(axis=1) < 0).mean() * 100:.1f} %")
```




```
Part des trajectoires avec un taux négatif à un moment : 29.6 %
```

Interprétation : les trajectoires partent de 2 % et remontent en moyenne vers θ ≈ 4,9 % en 1 à 2 ans. L'intervalle 5 %-95 % se stabilise ensuite (environ ±3,4 points autour de θ, car l'écart-type de long terme vaut $\sigma/\sqrt{2\kappa} \approx 2$ %) : c'est la conséquence du retour à la moyenne. Comme le modèle est gaussien et que le taux de départ est bas, **près de 30 % des trajectoires passent au moins une fois sous zéro** en 5 ans. C'est un défaut connu de Vasicek, aggravé ici par un σ élevé : un modèle qui empêche les taux négatifs (CIR) serait plus réaliste.

5.4 Validation : prix d'un zéro-coupon par Monte Carlo et par la formule fermée

Le prix d'un zéro-coupon qui paie 1 dans T ans est $P(0,T) = E\left[e^{-\int_0^T r_s ds}\right]$.

- **Monte Carlo** : pour chaque trajectoire, on calcule l'intégrale du taux (aire sous la courbe, méthode des trapèzes), puis la moyenne de $e^{-\text{intégrale}}$.
- **Formule fermée de Vasicek** : $P(0,T) = A(T)e^{-B(T)r_0}$, avec $B(T) = \frac{1-e^{-\kappa T}}{\kappa}$ et $A(T) = \exp\left[\left(\theta - \frac{\sigma^2}{2\kappa^2}\right)(B(T)-T) - \frac{\sigma^2 B(T)^2}{4\kappa}\right]$.

Si la simulation est correcte, les deux prix doivent converger quand le nombre de trajectoires augmente.

In [32]:
```
def prix_zc_vasicek(T, r0=R0, kappa=KAPPA, theta=THETA, sigma=SIGMA):
    # Formule fermée de Vasicek
    B = (1 - np.exp(-kappa * T)) / kappa
    A = np.exp((theta - sigma ** 2 / (2 * kappa ** 2)) * (B - T) - sigma
** 2 * B ** 2 / (4 * kappa))
    return A * np.exp(-B * r0)

def prix_zc_monte_carlo(trajectoires_r):
    # Moyenne de exp(-intégrale du taux court) ; renvoie le prix et son
erreur standard
    integrale = DT * (trajectoires_r[:, 1:] + trajectoires_r[:,
:-1]).sum(axis=1) / 2   # trapèzes
    actualisation = np.exp(-integrale)
    return actualisation.mean(), actualisation.std() /
```

```
np.sqrt(len(actualisation))

T_ZC = 2
prix_ferme = prix_zc_vasicek(T_ZC)

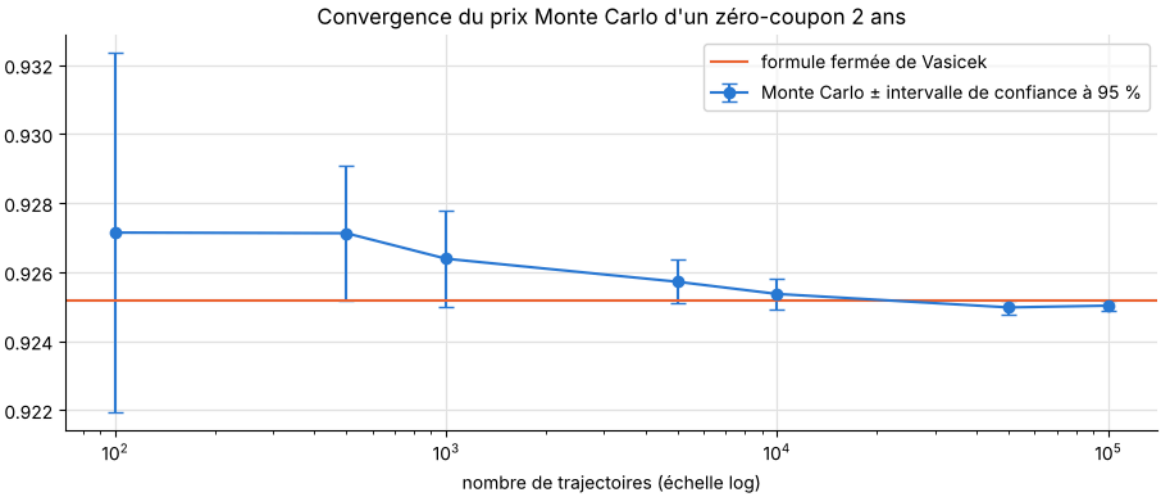
lignes = []
for n in [100, 500, 1000, 5000, 10000, 50000, 100000]:
    traj = simuler_vasicek(R0, KAPPA, THETA, SIGMA, horizon_ans=T_ZC,
nb_trajectoires=n, graine=1)
    p_mc, erreur_std = prix_zc_monte_carlo(traj)
    lignes.append({"trajectoires": n, "prix Monte Carlo": p_mc, "erreur
standard": erreur_std,
                   "écart à la formule (pb de prix)": (p_mc -
prix_ferme) * 10000,
                   "formule dans l'IC 95 % ?": abs(p_mc - prix_ferme) <
1.96 * erreur_std})
convergence = pd.DataFrame(lignes).set_index("trajectoires")
print(f"Prix formule fermée du zéro-coupon {T_ZC} ans :
{prix_ferme:.5f}")
pd.options.display.float_format = "{:.5f}".format
display(convergence)
pd.options.display.float_format = "{:.3f}".format
```

```
Prix formule fermée du zéro-coupon 2 ans : 0.92517
```

trajectoires	prix Monte Carlo	erreur standard	écart à la formule (pb de prix)	formule dans l'IC 95 % ?
100	0.92714	0.00266	19.72365	True
500	0.92713	0.00100	19.54850	True
1000	0.92639	0.00071	12.15279	True
5000	0.92572	0.00032	5.47114	True
10000	0.92537	0.00023	1.94034	True
50000	0.92498	0.00010	-1.96178	True
100000	0.92503	0.00007	-1.45537	False

In [33]:
```
fig, ax = plt.subplots(figsize=(11, 4))
ax.errorbar(convergence.index, convergence["prix Monte Carlo"],
yerr=1.96 * convergence["erreur standard"],
            color=BLEU, marker="o", capsize=4, label="Monte Carlo ±
intervalle de confiance à 95 %")
ax.axhline(prix_ferme, color=ORANGE, label="formule fermée de Vasicek")
ax.set_xscale("log")
ax.set_xlabel("nombre de trajectoires (échelle log)")
ax.set_title(f"Convergence du prix Monte Carlo d'un zéro-coupon {T_ZC}
ans")
ax.legend()
plt.show()

print("Erreur standard × racine(N) (doit être à peu près constante) :",
      (convergence["erreur standard"] *
np.sqrt(convergence.index)).round(4).tolist())
```




```
Erreur standard × racine(N) (doit être à peu près constante) : [0.0266, 0.
0223, 0.0224, 0.0228, 0.0228, 0.0229, 0.0228]
```

Interprétation :

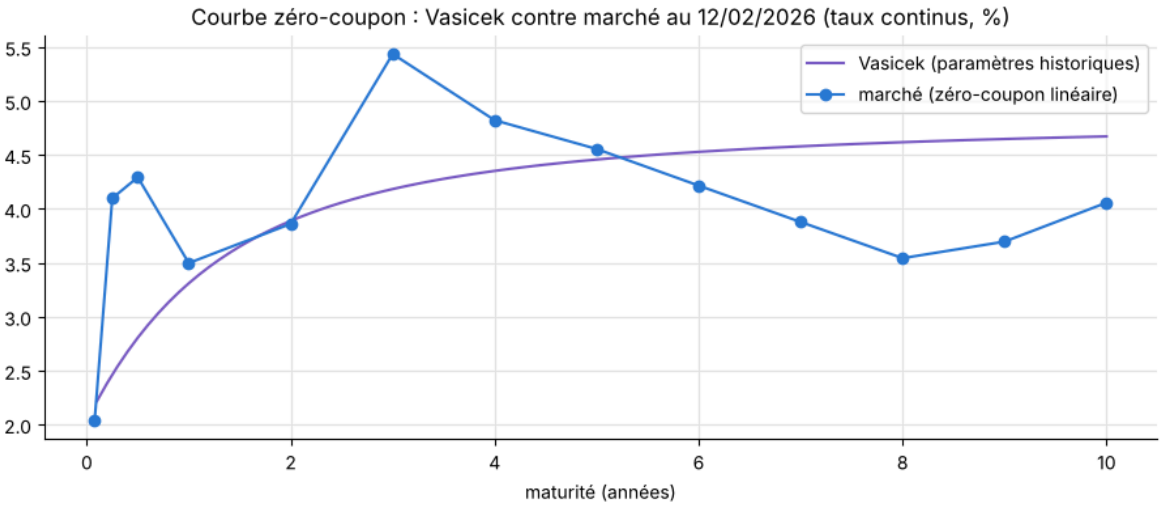
- Le prix Monte Carlo **converge vers la formule fermée** (0,92517) : l'écart passe de 20 pb de prix avec 100 trajectoires à moins de 2 pb avec 50 000 ou 100 000. La simulation est donc validée.
- À 100 000 trajectoires, l'écart (1,5 pb) vaut 2,1 erreurs standard, juste au-delà de l'intervalle à 95 %. C'est ce qu'on attend une fois sur vingt avec un intervalle à 95 %, et l'écart reste négligeable (0,016 % du prix).
- L'erreur standard diminue comme $1/\sqrt{N}$ (la colonne erreur standard × $\sqrt{N}$ est à peu près constante) : pour diviser l'erreur par 10, il faut **100 fois plus** de trajectoires. C'est la limite pratique du Monte Carlo.

5.5 Courbe de Vasicek et courbe de marché

La formule fermée donne une courbe zéro-coupon complète : $z(T) = -\ln P(0,T)/T$. On la compare à la courbe de marché.

In [34]:
```
T_courbe = np.linspace(0.1, 10, 100)
zc_vasicek = -np.log(prix_zc_vasicek(T_courbe)) / T_courbe
zc_marche_continu = np.log(1 + zc_lin)              # même convention
(taux continus)

fig, ax = plt.subplots(figsize=(11, 4))
ax.plot(T_courbe, zc_vasicek * 100, color=VIOLET, label="Vasicek
(paramètres historiques)")
ax.plot(zc_marche_continu.index, zc_marche_continu * 100, color=BLEU,
marker="o", label="marché (zéro-coupon linéaire)")
ax.set_title("Courbe zéro-coupon : Vasicek contre marché au 12/02/2026
(taux continus, %)")
ax.set_xlabel("maturité (années)")
ax.legend()
plt.show()
```



Interprétation : la courbe de Vasicek monte régulièrement de 2 % vers 4,5 % à 5 ans et 4,7 % à 10 ans. Elle ne reproduit pas la forme irrégulière de la courbe de marché. C'est normal : le modèle est calibré sur **l'historique** du taux court (probabilité « réelle »), pas sur les prix de marché (probabilité « risque-neutre »), et il n'a qu'un seul facteur. Pour valoriser des produits de taux de manière cohérente avec le marché, on utiliserait un modèle calé sur la courbe du jour (Hull-White, par exemple). Ici, Vasicek sert à **simuler des scénarios de taux** pour mesurer le risque.

5.6 VaR à 95 % et 99 % du portefeuille

Méthode :

1. On simule 10 000 valeurs du taux court dans **10 jours ouvrés** (2 semaines, l'horizon réglementaire de Bâle), avec le modèle de Vasicek.
2. On en déduit un choc sur la courbe zéro-coupon et on **revalorise** le portefeuille dans chaque scénario.
3. La **VaR à 95 %** est la perte dépassée dans seulement 5 % des scénarios (le 5e percentile des pertes et profits) ; la VaR à 99 % de même avec 1 %. On calcule aussi l'**Expected Shortfall** (perte moyenne au-delà de la VaR).

Comment transmettre le choc du taux court à la courbe ? Deux versions :

- **Vasicek pur** : dans le modèle, un choc Δr sur le taux court déplace le taux zéro-coupon de maturité T de $\Delta r \times B(T)/T$. Comme κ est élevé, les taux longs bougent beaucoup moins que le court (à 10 ans, environ 8 % du choc).
- **Choc parallèle (prudent)** : on applique le même Δr à toutes les maturités.

On ignore le passage du temps sur 2 semaines (coupons courus, rapprochement de l'échéance) : on mesure uniquement l'effet des taux.

In [35]:
```
HORIZON = 2 / 52            # 10 jours ouvrés
NB_SCENARIOS = 10000

r_horizon = simuler_vasicek(R0, KAPPA, THETA, SIGMA,
horizon_ans=HORIZON, nb_trajectoires=NB_SCENARIOS, graine=7)[:, -1]
delta_r = r_horizon - R0
```

```
T_grille = zc_lin.index.values
B_sur_T = (1 - np.exp(-KAPPA * T_grille)) / (KAPPA * T_grille)    # part
du choc transmise à chaque maturité

def valeur_portefeuille(zc):
    return sum(prix(titre, zc) * NOMINAL / 100 for _, titre in
portefeuille.iterrows())

valeur_0 = valeur_portefeuille(zc_lin)
pnl_vasicek = np.array([valeur_portefeuille(zc_lin + d * B_sur_T) -
valeur_0 for d in delta_r])
pnl_parallele = np.array([valeur_portefeuille(zc_lin + d) - valeur_0 for
d in delta_r])

def var_es(pnl, niveau):
    seuil = np.percentile(pnl, 100 - niveau)
    return -seuil, -pnl[pnl <= seuil].mean()

lignes = {}
for nom, pnl in [("Vasicek pur", pnl_vasicek), ("Choc parallèle",
pnl_parallele)]:
    var95, es95 = var_es(pnl, 95)
    var99, es99 = var_es(pnl, 99)
    lignes[nom] = {"VaR 95 % (M MRU)": var95, "VaR 99 % (M MRU)": var99,
                   "ES 95 % (M MRU)": es95, "ES 99 % (M MRU)": es99,
                   "VaR 99 % (% valeur)": var99 / valeur_0 * 100}
table_var = pd.DataFrame(lignes).T

print(f"Variation du taux court sur 10 jours : moyenne {delta_r.mean() *
10000:.1f} pb, écart-type {delta_r.std() * 10000:.1f} pb")
table_var
```

```
Variation du taux court sur 10 jours : moyenne 13.7 pb, écart-type 62.4 pb
```

Out[35]:

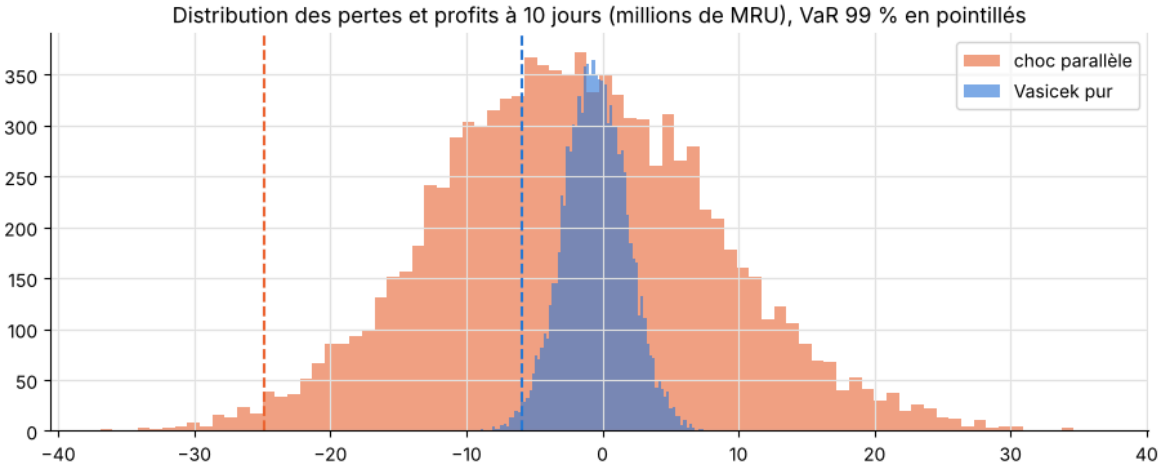
	VaR 95 % (M MRU)	VaR 99 % (M MRU)	ES 95 % (M MRU)	ES 99 % (M MRU)	VaR 99 % (% valeur)
Vasicek pur	4.360	5.917	5.325	6.682	0.956
Choc parallèle	18.525	24.871	22.456	27.928	4.016

In [36]:

```
# Contrôle avec une formule simple (choc parallèle) : VaR ≈ z × DV01 ×
écart-type du choc en pb
for niveau, z in [(95, 1.645), (99, 2.326)]:
    var_formule = (z * delta_r.std() * 10000 + delta_r.mean() * 10000) *
dv01_ptf
    print(f"VaR {niveau} % approchée par le DV01 : {var_formule:.2f} M
MRU")

fig, ax = plt.subplots(figsize=(11, 4))
ax.hist(pnl_parallele, bins=80, color=ORANGE, alpha=0.6, label="choc
parallèle")
ax.hist(pnl_vasicek, bins=80, color=BLEU, alpha=0.6, label="Vasicek
pur")
for nom, pnl, couleur in [("Vasicek", pnl_vasicek, BLEU), ("parallèle",
pnl_parallele, ORANGE)]:
    ax.axvline(-var_es(pnl, 99)[0], color=couleur, linestyle="--")
ax.set_title("Distribution des pertes et profits à 10 jours (millions de
MRU), VaR 99 % en pointillés")
ax.legend()
plt.show()
```

```
VaR 95 % approchée par le DV01 : 19.08 M MRU
VaR 99 % approchée par le DV01 : 26.05 M MRU
```



Interprétation :

- À 99 % et 10 jours, la VaR est d'environ **5,9 M MRU** (1 % de la valeur) en « **Vasicek pur** » et d'environ **24,9 M MRU** (4 % de la valeur) en « **choc parallèle** ». La première est bien plus faible : dans le modèle, les taux longs bougent peu, alors que c'est justement sur les maturités longues que le portefeuille est sensible (OT 10 ans). C'est un **risque de modèle** important : un seul facteur, calibré sur un taux à 4 semaines, ne dit rien de fiable sur les taux à 10 ans.
- La VaR parallèle est cohérente avec le calcul simple « z × DV01 × volatilité du choc » (26 M MRU contre 24,9 M MRU à 99 %). La petite différence vient de la convexité, que l'approximation linéaire ignore.
- La moyenne des pertes et profits est **négative** : le modèle prévoit une remontée du taux court vers θ.
- Les chiffres sont **probablement surestimés pour le court terme** (σ gonflé par le bruit d'adjudication) et **sous-estimés pour le long terme** dans la version Vasicek pur. Pour la gestion des risques, on retiendrait la version parallèle, plus prudente, en la complétant par les scénarios de la section 4.

6. Limites : une courbe construite à partir des adjudications

Les taux du fichier sont des **taux d'adjudication** (marché primaire), faute de marché secondaire liquide où les titres s'échangeraient tous les jours. Cela a plusieurs conséquences pour la fiabilité de la courbe :

1. **Des points datés et pas synchronisés.** Chaque maturité n'est mise à jour que lorsqu'elle est émise. Le 12/02/2026, le taux à 10 ans date d'août 2025, celui à 3 ans de janvier 2026, celui à 4 semaines de la dernière adjudication. On relie donc des taux observés **à des dates différentes**, dans des conditions monétaires différentes : la courbe n'est pas une photographie du marché à une date donnée.
2. **Des taux qui reflètent une seule séance.** Un taux d'adjudication dépend de la demande des banques ce jour-là, de leur liquidité et du montant que le Trésor

veut lever. D'où les sauts de 1,5 à 2 points d'une adjudication à l'autre, qu'aucun arbitrage ne vient corriger.

3. **Une courbe peu fiable entre les points et aux extrémités.** Les maturités 6 et 7 ans n'existent pas ; les maturités longues ne reposent que sur une émission chacune. Le choix de la méthode d'interpolation change le résultat de plusieurs dizaines de points de base exactement là où il n'y a pas de données.
4. **Une dynamique impossible à mesurer correctement.** Avec des taux reportés, la plupart des variations hebdomadaires sont nulles : l'ACP ne trouve pas de facteur commun, la volatilité est mal estimée (trop faible sur les maturités longues, gonflée par les sauts sur les courtes), et les mesures de risque (VaR, Vasicek) en héritent.
5. **Pas de prix de marché pour valoriser.** En l'absence de marché secondaire, la valeur d'un titre détenu par une banque est une valeur de **modèle** (actualisation sur la courbe), pas un prix auquel on pourrait vendre. Le risque de liquidité (ne pas pouvoir revendre avant l'échéance, ou seulement avec une forte décote) n'apparaît dans aucun des calculs.

Lien avec le développement du marché secondaire (thème de mon mémoire) : un marché secondaire actif apporterait exactement ce qui manque à cette courbe :

- des **prix quotidiens** pour chaque titre, donc une courbe mise à jour chaque jour sur toutes les maturités, et non une suite d'adjudications isolées ;
- l'**arbitrage** entre maturités voisines, qui lisserait les irrégularités (le pic à 3 ans ou le creux à 1 an ne survivraient pas si des investisseurs pouvaient acheter et vendre librement) ;
- des **variations de taux corrélées** entre maturités, qui rendraient l'ACP, la volatilité et la VaR beaucoup plus fiables ;
- une **référence de prix** pour valoriser les portefeuilles des banques et servir de base à d'autres financements (obligations d'entreprises, prêts à taux variable).

Les leviers classiques pour y parvenir : des **spécialistes en valeurs du Trésor** (banques qui s'engagent à coter des prix d'achat et de vente), un calendrier d'émission régulier et publié, la **réouverture** de lignes existantes plutôt que de nouvelles émissions (moins de titres, mais plus gros et plus liquides), des opérations de **pension (repo)** sur titres publics et une plateforme de négociation avec publication des prix. Tant que ces conditions ne sont pas réunies, la courbe des taux souveraine en MRU doit être lue comme un **indicateur des conditions d'émission du Trésor**, pas comme une courbe de marché au sens strict.

7. Synthèse des résultats

Question	Résultat
Données	196 semaines (01/2022-02/2026). La courbe s'allonge progressivement : 105 semaines avec au moins 5 maturités, 25 seulement avec les 11. Les taux sont les derniers taux d'adjudication, reportés d'une semaine à l'autre (une seule valeur sur toute la période pour les 4 à 10 ans).
Forme de la courbe au 12/02/2026	Très irrégulière : 2,1 % à 4 semaines, creux à 3,6 % à 1 an, pic à 5,5 % à 3 ans, 3,7-4,2 % à 8-10 ans ; tous les titres sous le taux directeur (6 %).
Meilleure méthode	Interpolation linéaire (méthode BCM) : meilleure erreur hors échantillon (82 pb, contre 89 pb pour Nelson-Siegel et 121 pb pour la spline). La spline oscille entre les points. Nelson-Siegel lisse (43 pb d'erreur dans l'échantillon) et résume la courbe en 3 facteurs.

Question	Résultat
Stabilité de Nelson-Siegel	Paramètres instables avec λ libre ; fixer λ réduit les sauts d'environ 25 %. Les β ne deviennent stables qu'après l'arrivée des maturités longues (mars et août 2025).
Zéro-coupon et forwards	Bootstrapping des taux au pair : zéro-coupons proches des taux au pair ; facteur d'actualisation à 10 ans de 0,667. Les forwards amplifient les irrégularités (de 1,2 % à 9 %) : ce ne sont pas de vraies anticipations.
ACP	Sur les variations brutes des bons, pas de facteur commun (corrélations proches de 0, chaque composante = une maturité). Sur la courbe lissée : niveau 75 %, pente 19 %, courbure 7 %.
Portefeuille (600 M MRU de nominal)	Valeur 619 M MRU, duration modifiée 2,65, convexité 16,8, DV01 ≈ 164 000 MRU par pb. Les titres émis en 2024 à 6-7 % valent plus que 100 grâce à la baisse des taux.
Chocs	+100 pb parallèle : -15,9 M MRU (-2,6 %), bien approché par duration + convexité. Pentification : -0,9 M MRU ; aplatissement : +1,1 M MRU (exposition à l'OT 10 ans).
Vasicek (taux à 4 semaines)	κ = 1,32 (demi-vie 6 mois, mais retour à la moyenne non significatif), θ = 4,9 %, σ = 3,3 % (gonflée par le bruit d'adjudication). Près de 30 % des trajectoires passent sous zéro.
Validation Monte Carlo	Le prix Monte Carlo du zéro-coupon 2 ans converge vers la formule fermée (0,92517) ; l'erreur décroît en $1/\sqrt{N}$.
VaR 10 jours	99 % : 5,9 M MRU (Vasicek pur) à 24,9 M MRU (choc parallèle, prudent). L'écart entre les deux est un risque de modèle.
Message principal	Les outils standard fonctionnent, mais la qualité des résultats est limitée par les données : une courbe d'adjudications, non synchronisée et sans marché secondaire, donne des forwards, des facteurs et des volatilités peu fiables.